

Enhancing the DATF Technique in Differential-Linear Cryptanalysis

Cheng Che^{1,2} and Tian Tian^{1(✉)}

¹ Information Engineering University, Zhengzhou 450001, China

² State Key Laboratory of Cryptology, P.O. Box 5159, Beijing 100878, China
che_cheng@126.com, tiantian_d@126.com

Abstract. Differential-linear cryptanalysis was introduced by Langford and Hellman at CRYPTO’94 and has been an important cryptanalysis method against symmetric-key primitives. The current primary framework for constructing differential-linear distinguishers involves dividing the cipher into three parts: the differential part E_0 , the middle connection part E_m , and the linear part E_1 . This framework was first proposed at EUROCRYPT 2019, where DLCT was introduced to evaluate the differential-linear bias of E_m over a single round. Recently, the TDT method and the generalized DLCT method were proposed at CRYPTO 2024 to evaluate the differential-linear bias of E_m covering multiple rounds. Unlike the DLCT framework, the DATF technique could also handle E_m covering more rounds.

In this paper, we enhance the DATF technique in differential-linear cryptanalysis from three perspectives. First, we improve the precision of differential-linear bias estimation by introducing new transitional rules, a backtracking strategy, and a partitioning technique to DATF. Second, we present a general bias computation method for Boolean functions that significantly reduces computational complexity compared to the exhaustive search used by Liu et al. in the previous DATF technique. Third, we propose an effective method for searching for differential-linear distinguishers with good biases based on DATF. Additionally, the bias computation method has independent interests with a wide application in other cryptanalysis methods, such as differential cryptanalysis and cube attacks. Notably, all these enhancements to DATF are equally applicable to HATF. To show the validity and versatility of our new techniques, we apply the enhanced DATF to the NIST standard Ascon, the AES finalist Serpent, the NIST LWC finalist Xoodyak, and the eSTREAM finalist Grain v1. In all applications, we either present the first differential-linear distinguishers for more rounds or update the best-known ones.

Keywords: Differential-Linear Cryptanalysis · DATF · Ascon · Serpent · Xoodyak · Grain v1

1 Introduction

Differential cryptanalysis [7] and linear cryptanalysis [28] are the two most well-known techniques for analyzing symmetric-key cryptographic primitives. Most

modern cryptographic primitives are designed to withstand these two cryptanalysis methods by avoiding the generation of long, high-probability differential or linear trails. In [22], an efficient attack method known as differential-linear (DL) cryptanalysis, which combines differential and linear cryptanalysis, was proposed by Langford and Hellman in 1994. DL cryptanalysis has been shown to outperform the two fundamental attack methods against certain ciphers, including the AES finalist Serpent [6], the CAESAR candidate ICEPOLE [20], the ARX cipher ChaCha [34], and COCONUT98 [5], etc.

The main idea of DL cryptanalysis is to decompose the cryptographic algorithm E into two sub-ciphers E_0 and E_1 , i.e., $E = E_1 \circ E_0$. It is expected that E_0 has a high-probability differential $\Delta_I \xrightarrow{p} \Delta_O$ and E_1 has a high-bias linear approximation $\lambda_I \xrightarrow{q} \lambda_O$. These two parts are connected to construct a differential-linear distinguisher, that is,

$$\Pr[\lambda_O \cdot E(P) \oplus \lambda_O \cdot E(P \oplus \Delta_I)] = \frac{1}{2} + (-1)^{\Delta_O \cdot \lambda_I} 2pq^2.$$

However, two assumptions are required for the bias of the distinguisher to be the estimated value $2pq^2$: (1) the probability of $\lambda_I \cdot E_1(P) \oplus \lambda_I \cdot E_1(P \oplus \Delta_I) = \lambda_I \cdot \Delta_O$ is $1/2$ when $E_1(P) \oplus E_1(P \oplus \Delta_I) \neq \Delta_O$, and (2) E_0 and E_1 are statistically independent. Relaxing these two foundational assumptions is one of the interesting research directions for DL cryptanalysis.

Regarding Assumption (1), Lu [27] first observed that the technique applied to the first sub-cipher should be truncated differentials rather than classical differentials. This is because the inner product of λ and Δ is only related to the active bit positions of λ , so the values at the remaining positions in Δ can be arbitrary. Subsequently, in 2014, Blondeau et al. [8] conducted a more detailed analysis of the middle layer of the DL distinguisher. They introduced the concept of a differential-linear hull and provided an exact expression for the bias based solely on Assumption (2). This expression is a theoretical result; in practice, evaluating the bias from this expression is difficult.

The dependency between the two parts of DL distinguishers remained unaddressed until EUROCRYPT 2019. Rather than assuming that the two parts are independent, Bar-On et al. [2] introduced the *Differential-Linear Connectivity Table* (DLCT) to characterize the dependency between the differential and linear part. Based on the innovative concept, the cipher E is decomposed into three parts: $E = E_1 \circ E_m \circ E_0$, where E_0 is covered by the differential trail, E_m is covered by the DLCT, and E_1 is covered by the linear approximation. A significant limitation is that DLCT can only be applied to a single S-layer. More recently, at CRYPTO 2024, Peng et al. [32] and Hadipour et al. [17] independently proposed methods for extending DLCT into multiple rounds. In [32], Peng et al. proposed the *Truncated Difference Distribution Table* (TDT), which can transform the computation of the differential-linear bias into probability computation involving a series of truncated differentials. In [17], Hadipour et al. analyzed E_m from the perspective of boomerang analysis and introduced a series of new tables to facilitate the formulation of the dependency between

E_0 and E_1 across multiple rounds. In addition, they introduced an automated search method for high-quality DL distinguishers based on CP/MILP.

Unlike the DLCT framework, Liu et al. [26] proposed a purely algebraic method to estimate the differential-linear bias at CRYPTO 2021. Theoretically, the bias of a DL distinguisher can be determined by the algebraic normal form (ANF) of the output bits, with the input bits as variables. However, it is computationally infeasible to obtain ANFs for a cryptographic algorithm. Consequently, they introduced a novel technique called the *Differential Algebraic Transitional Form* (DATF) to describe the iterative process of round functions. By computing the algebraic transitional form (ATF), the DATF of the output bits is obtained to estimate the bias of the DL distinguisher. Later, at ASIACRYPT 2023, Hu et al. extended the DATF technique to high-order differential-linear (HDL) cryptanalysis and proposed the *Higher-order Algebraic Transitional Form* (HATF).

Motivation. With the development of DLCT and its extensions, the DLCT framework has become increasingly mature. On the other hand, estimating the differential-linear bias from an algebraic perspective has many advantages, such as high efficiency, simplicity, and wide applications. The proposal of DATF helps to address weaknesses in the DLCT framework for some cryptographic algorithms that are either unsuitable for table construction or have an excessive number of active S-boxes. Specifically, DATF is more applicable to bit-based cryptographic algorithms, including block ciphers, authenticated encryption algorithms, stream ciphers, and hash functions. According to our analysis, the two issues that are closely related to the effectiveness of the DATF technique are (1) whether the bias of the DATF well approximates the exact bias and (2) how to fast compute the bias of the DATF. Another problem that needs to be considered in DL cryptanalysis is (3) how to efficiently search DL distinguishers based on DATF. In this paper, we aim to study these three issues. In fact, the second issue can also be expressed as how to quickly compute the exact bias of a given ANF, which is of concern to many cryptanalysis methods such as differential cryptanalysis and cube attacks.

Our contributions. In this paper, we enhance the DATF technique in DL cryptanalysis. The enhanced DATF demonstrates superior performance compared to the original DATF in terms of efficiency when searching for DL distinguishers, as well as in the speed and precision of bias estimation. Our enhancements mainly come from the following three aspects, and all these enhancements to DATF are equally applicable to HATF.

1. **Improvement on the precision of DATF.** Three methods are discussed to reduce the gap between the bias of DATF and that of the differential function, thereby improving the precision of the DATF technique. First, new transitional rules are proposed to avoid some common situations where transitional variables have dependencies. Second, a backtracking strategy is proposed for isolated variables to prevent the introduction of new errors.

Finally, the partitioning technique is introduced to simplify the DATF and clarify its algebraic structure for better representation of bias.

2. **A general bias computation method.** For a Boolean function without a special structure, it may be necessary to enumerate all the variables to obtain the bias. We propose a general method for computing bias based on a new lemma. This method can significantly reduce the number of variables that need to be enumerated. In DL cryptanalysis, it can speed up the estimation of the differential-linear bias based on DATF. In our experiments, compared to the method in [26], the computation time of the new method is reduced by factors of tens or even hundreds. The more variables the expression contains and the more complex the structure, the better the improvement effect of the new method.
3. **An effective method for finding good distinguishers.** In the previous exhaustive search, the attacker needed to estimate the biases for all input differences and output masks, which led to a very high complexity. To address this issue, we propose an efficient method based on DATF for finding good DL distinguishers. This new method can effectively remove a large number of differential-linear pairs with small biases, which has been experimentally verified.

By applying the enhanced DATF to several cryptographic algorithms, we provide more precise bias estimates and obtain some new distinguishers that are either longer or better, as detailed in Table 1.

- For Ascon, we demonstrate that the 6-round DL distinguisher presented in [32] is invalid and provide the first 6-round CDL distinguisher.
- For Serpent, we propose a best 9-round DL distinguisher.
- For Xoodyak, we propose a 2nd-order HDL distinguisher for the 6-round Xoodyak under the related-key model. This is the first distinguisher for the 6-round Xoodyak.
- For Grain v1, we provide a 126-round CDL distinguisher that can generate a bias of $2^{-17.4}$ by adding 25 conditions.

Additionally, based on the new distinguishers, we present some key recovery attacks, as shown in Table 2.

- For Ascon, we propose the first differential-linear attack on 6-round Ascon and the best 4-round and 5-round key recovery attacks.
- For Serpent, we propose a new key recovery attack on 11-round Serpent. This attack is the best in terms of both time complexity and data complexity.
- For Xoodyak, we present a key recovery attack on the 6-round Xoodyak under the related-key model. This is the first differential-linear cryptanalysis for the 6-round Xoodyak.
- For Grain v1, we propose a key recovery attack on 126-round Grain v1, which represents the highest number of rounds analyzed to date.

We implemented the enhanced DATF technique in C++; the source codes are provided in the git repository <https://github.com/chengcor/Enhanced-DATF>.

Table 1. The (high-order) differential-linear and differential bias

Cipher	Rounds	Bias		Type	Method	Source
		Expr.	Theory			
Ascon	5	$2^{-4.54}$	$2^{-5.42}$	CDL	DATF	[26]
			2^{-5}		Enhanced DATF	Sect. 6.1
		$2^{-2.99}$	2^{-3}	CDL	Enhanced DATF	Sect. 6.1
	6	2^{-22}	$2^{-25.97}$	CHDL	HATF	[19]
		-	$2^{-22.43*}$	DL	TDT	[32]
		$2^{-15.91}$	2^{-16}	CDL	Enhanced DATF	Sect. 6.1
Serpent	9	$2^{-55.42}^\dagger$	$2^{-55.43}$	DL	GDLCT	[17]
			$2^{-55.40}$		Enhanced DATF	Sect. 6.2
		$2^{-55.33}^\dagger$	$2^{-55.33}$	DL	TDT	[32]
			$2^{-55.26}$		Enhanced DATF	Sect. 6.2
		$2^{-52}^\dagger$	2^{-52}	DL	TDT	[32]
			2^{-52}		Enhanced DATF	Sect. 6.2
		$2^{-51.89}^\dagger$	$2^{-51.95}$	DL	GDLCT	[17]
Xoodoo	4		$2^{-9.67}$	DL	Experimental	[15]
			$2^{-9.68}$		HATF	[19]
			$2^{-9.68}$		Enhanced DATF	Sect. 6.3
		$2^{-5.72}$	$2^{-5.72}$	HDL	HATF	[19]
	5		$2^{-5.72}$		Enhanced DATF	Sect. 6.3
		2^{-1}	2^{-1}	HDL	HATF	[19]
			2^{-1}		Enhanced DATF	Sect. 6.3
	6	$2^{-9.36}^\ddagger$	-	DL	Experimental	[15]
			2^{-10}		HATF	[19]
			$2^{-9.36}$		Enhanced DATF	Sect. 6.3
Grain v1	120		2^{-45}	HDL	HATF	[19]
			$2^{-42.88}$		Enhanced DATF	Sect. 6.3
		-	$2^{-48.47}^\ddagger$	HDL	Enhanced DATF	Sect. 6.3
	125	$2^{-12.8}$	-	CD	Experimental	[24]
			$2^{-13.39}$		DATF	[26]
			$2^{-13.11}$		Enhanced DATF	Sect. 6.4
Grain v1	126	$2^{-17.4}$	$2^{-20.77}$	CD	DATF	[26]
			$2^{-17.16}$		Enhanced DATF	Sect. 6.4
	126	$2^{-17.4}$	$2^{-17.16}$	CD	Enhanced DATF	Sect. 6.4

* It is not valid, see Section 6.1.

† Only the middle part E_m is experimented.

‡ This distinguisher is under the related-key model.

Organization. The rest of the paper is organized as follows. In Section 2, we briefly introduce the existing methods for evaluating bias. In Section 3, we discuss methods to improve the precision of bias estimation. In Section 4, a novel and effective bias computation method is proposed, which speeds up the estimation

Table 2. Key recovery attacks on Ascon, Serpent, Xoodyak, and Grain v1

Cipher	Rounds	Time	Data	Space	Method	Source
Ascon	4	2^{18}	2^{18}	neg.	DL	[12]
		2^{11}	2^{11}	neg.	Cond. DL	Sect. 6.1
	5	2^{36}	2^{36}	neg.	DL	[12]
		2^{35}	2^{35}	neg.	Cube-like	[12]
		2^{26}	2^{26}	neg.	Cond. DL	[26]
		2^{24}	2^{24}	neg.	Cond. Cube	[25]
		2^{22}	2^{22}	neg.	Cond. HDL	[19]
		2^{20}	2^{20}	neg.	Cond. DL	Sect. 6.1
	6	2^{40}	2^{40}	neg.	Cond. Cube	[25]
		2^{47}	2^{47}	neg.	Cond. DL [†]	Sect. 6.1
Serpent	11	$2^{135.7}$	$2^{121.8}$	2^{76}	DL	[14]
		$2^{137.7}$	$2^{113.7}$	2^{99}	DL	[14]
		$2^{125.7}$	$2^{125.7}$	2^{99}	DL	[26]
		$2^{125.34}$	$2^{124.8}$	2^{60}	DL	Sect. 6.2
	12	$2^{253.8}$	$2^{125.8}$	$2^{125.8}$	Multdim-lim	[31,29,10]
		2^{242}	$2^{125.8}$	2^{236}	Multdim-lim	[31,29,10]
		2^{251}	2^{127}	2^{127}	DL	[26,10]
		$2^{214.36}$	$2^{125.16}$	$2^{125.16}$	Linear	[16]
		$2^{210.36}$	$2^{126.30}$	$2^{125.16}$	Linear	[16]
Xoodyak	5	2^{22}	2^{22}	neg.	DL [‡]	[15]
		2^{70}	2^{70}	neg.	Cond. HDL	[19]
	6	$2^{94.3}$	$2^{94.3}$	neg.	Cond. HDL [‡]	Sect. 6.3
		$2^{43.8}$	$2^{43.8}$	neg.	Cond. Cube	[35]
Grain v1	120	2^{52}	$2^{29.75}$	neg.	Cond. D	[24]
	125	2^{60}	2^{52}	neg.	Cond. D	[26]
	126	2^{60}	2^{53}	neg.	Cond. D	Sect. 6.4

[†] The attack is applicable to Ascon-AEAD128 and Ascon-128a.

[‡] The attack is under the related-key model.

of the DL bias. Then, we provide a method that can efficiently search for good DL distinguishers in Section 5. Based on the contributions of the previous three sections, we apply the enhanced DATF to several cryptographic algorithms in Section 6. Finally, Section 8 concludes the paper.

2 Preliminaries

2.1 Boolean Function

Let $\mathbb{F}_2$ denote the binary field and $\mathbb{F}_2^n$ denote the vector space of dimension n over $\mathbb{F}_2$. An n -variable Boolean function is a mapping from $\mathbb{F}_2^n$ to $\mathbb{F}_2$. A Boolean function f can be uniquely represented as a multivariable polynomial over $\mathbb{F}_2$,

i.e.,

$$f(x_0, x_1, \dots, x_{n-1}) = \bigoplus_{\mathbf{c}=(c_0, \dots, c_{n-1}) \in \mathbb{F}_2^n} a_{\mathbf{c}} \prod_{i=0}^{n-1} x_i^{c_i}, \quad a_{\mathbf{c}} \in \mathbb{F}_2,$$

which is called the algebraic normal form (ANF) of f , and $\prod_{i=0}^{n-1} x_i^{c_i}$ is called a term of f if $a_{\mathbf{c}} = 1$. We denote the set of all terms in f by $T(f)$, and the set of variables that appear in f by $\text{var}(f)$. The bias ε of f is defined as $\varepsilon = \Pr[f = 0] - \frac{1}{2}$, and we say that f is balanced if $\varepsilon = 0$.

For the Boolean polynomial $f(x_0, x_1, \dots, x_{n-1})$, given a variable x_i , $f|_{x_i=1}$ and $f|_{x_i=0}$ are defined by

$$\begin{aligned} f|_{x_i=1} &\triangleq f(x_0, \dots, x_{i-1}, 1, x_{i+1}, \dots, x_{n-1}), \\ f|_{x_i=0} &\triangleq f(x_0, \dots, x_{i-1}, 0, x_{i+1}, \dots, x_{n-1}). \end{aligned}$$

The polynomial f can be uniquely represented by $f = x_i \cdot D_{x_i} f \oplus f|_{x_i=0}$, where $D_{x_i} f$ is the partial derivative of f with respect to the variable x_i . A variable x_i is called isolated if it appears and only appears in the linear part of the ANF of f , i.e., $D_{x_i} f = 1$.

2.2 The Differential-Linear Cryptanalysis

Differential-linear cryptanalysis was proposed by Langford and Hellman at CRYPTO 1994 [22]. For a cipher E , given the input difference Δ_i and output mask λ_o , the differential-linear bias ε can be derived from the following equation

$$\Pr(\lambda_o \cdot E(x) \oplus \lambda_o \cdot E(x \oplus \Delta_i) = 0) = \frac{1}{2} + \varepsilon.$$

Similar to other attacks like differential attacks or linear attacks, additional rounds can be added before and/or after the DL distinguisher. Then, the correct key involved in these additional rounds can be identified by evaluating the DL distinguisher. According to [33], assuming that the attacker uses N input differential pairs and guesses k bits of key, the success probability P_s of DL cryptanalysis can be calculated as

$$P_s = \Phi(2\sqrt{N}|\varepsilon| - \Phi^{-1}(1 - 2^{-k-1})), \quad (1)$$

where Φ is the cumulative distribution function of the standard normal distribution.

In [22], the DL distinguisher is constructed by connecting the differential and linear trails. Assuming that the cryptographic algorithm E is decomposed into E_0 and E_1 , as shown in Fig. 1(a), where E_0 has a differential characteristic $\Delta_i \rightarrow \Delta_o$ with a probability p and E_1 has a linear approximation $\lambda_i \rightarrow \lambda_o$ with a bias q , the bias of the DL distinguisher after combination can be calculated as

$$\Pr(\lambda_o \cdot E(x) \oplus \lambda_o \cdot E(x \oplus \Delta_i)) = p \cdot \left(\frac{1}{2} + 2q^2\right) + (1-p) \cdot \frac{1}{2} = \frac{1}{2} + 2pq^2.$$

It is worth noting that this bias is only valid when the two assumptions previously described hold.

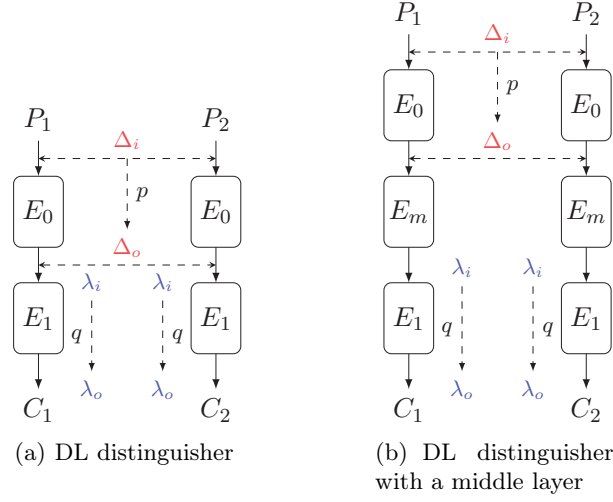


Fig. 1. The structure of DL distinguishers

2.3 The DLCT Framework

At EUROCRYPT 2019, Bar-On et al. [2] introduced a new tool, the Differential-Linear Connectivity Table (DLCT), to deal with the dependency between E_0 and E_1 . They proposed the DLCT framework with three parts, as illustrated in Fig. 1(b), where E_0 and E_1 are connected by DLCT.

Definition 1. [2] Let $S : \{0, 1\}^n \rightarrow \{0, 1\}^m$ be a vectorial Boolean function. The DLCT of S is a $2^n \times 2^m$ table whose rows correspond to input differences to S and whose columns correspond to bit masks of outputs of S . Formally, for $\Delta \in \{0, 1\}^n$ and $\lambda \in \{0, 1\}^m$, we can compute

$$\text{DLCT}_S(\Delta, \lambda) \triangleq |\{X \mid \lambda \cdot S(X) = \lambda \cdot S(X \oplus \Delta)\}| - 2^{n-1}$$

and

$$\overline{\text{DLCT}}_S(\Delta, \lambda) \triangleq \frac{\text{DLCT}_S(\Delta, \lambda)}{2^n}.$$

Given a differential-linear pair (Δ, λ) , the bias of $\Delta \xrightarrow{E_m} \lambda$ is $\overline{\text{DLCT}}_{E_m}(\Delta, \lambda)$. Then, the bias of the DL distinguisher is

$$\varepsilon = 4 \cdot p \cdot \overline{\text{DLCT}}_{E_m}(\Delta, \lambda) \cdot q^2. \quad (2)$$

Considering the in-depth analysis in [9], given the input difference Δ_i and the output linear mask λ_o , the exact bias of the DL distinguisher is

$$\varepsilon = \sum_{\Delta, \lambda} 4 \cdot \Pr(\Delta_i \xrightarrow{E_0} \Delta) \cdot \overline{\text{DLCT}}_{E_m}(\Delta, \lambda) \cdot (q_{\lambda \xrightarrow{E_1} \lambda_o})^2.$$

In practice, attackers usually find a trail with the highest probability to evaluate this bias. In 2024, the DLCT framework has been further strengthened and the middle part E_m has been able to handle the case of multiple rounds [32,17].

2.4 The DATF Technique

Recently, Liu et al. [26] studied DL cryptanalysis from an algebraic perspective, opening a new avenue for both differential and DL cryptanalysis. For a cipher E , the bias of the DL distinguisher $\Delta_i \xrightarrow{E} \lambda_o$ is related to the differential bias of the Boolean function $f(X) = \lambda_o \cdot E(X)$. Since computing the ANF of the difference function $f(X) \oplus f(X \oplus \Delta_i)$ is computationally infeasible, Liu et al. proposed computing the algebraic transitional forms (ATF) for the cipher E . Specifically, they compute the differential algebraic transitional forms (DATF) of the internal and output bits and then estimate the bias of these DATFs. This approach requires that the DATF be a Boolean expression with a bias similar to that of $f(X) \oplus f(X \oplus \Delta_i)$ and possess a simple structure.

Next, we will briefly introduce how to compute the DATF for an iterative cipher. Given the round function R of the cipher and an input difference Δ_i , we first initialize $Y^{(0)} = X \oplus x\Delta_i$, where x is a binary variable and X is a binary variable vector. After the round function R , the internal state is $Y^{(1)} = Y'^{(1)} \oplus xY''^{(1)}$, where $Y'^{(1)} = Y^{(1)}|_{x=0}$, $Y''^{(1)} = D_x Y^{(1)}$. Then replace $Y'^{(1)}$ and $Y''^{(1)}$ with new variables $\alpha^{(1)}$ and $\beta^{(1)}$ to obtain ATFs $\alpha^{(1)} \oplus x\beta^{(1)}$. Repeat this process until the ATFs corresponding to the output bits are obtained, and record all substitute variables and their corresponding expressions in the set Q . The diagram of the procedure is depicted in Fig. 2. The complexity of this step is $\mathcal{O}(2^d r t_R)$, where t_R represents the complexity of computing the ANF of the round function R , and d is the algebraic degree of R . For a detailed evaluation of this complexity, please refer to [26].

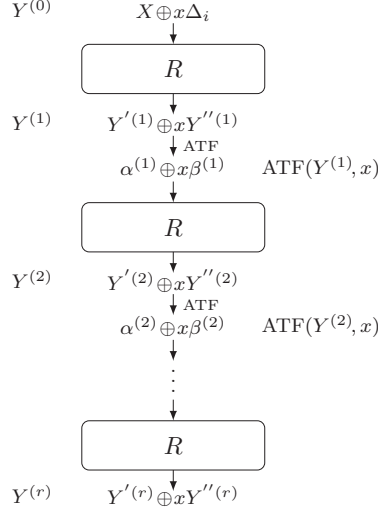
Given the linear mask λ_o , the bias of the DL distinguisher will be estimated using the obtained ATF expressions, as shown in Algorithm 1. In Algorithm 1, Liu et al. used a procedure `Bias()` to compute the bias given an algebraic expression in binary variables, which is based on the following lemma.

Lemma 1. [26] *Given a vectorial Boolean function $f : \mathbb{F}_2^n \rightarrow \mathbb{F}_2^n$ and n input bits $x_0, x_1, \dots, x_{n-1}$ with biases $\varepsilon_0, \varepsilon_1, \dots, \varepsilon_{n-1}$ respectively. Under the assumption that all inputs are independent, the bias of f is*

$$\text{Bias}(f) = \sum_{\substack{x_0, x_1, \dots, x_{n-1} \\ s.t. f(x_0, \dots, x_{n-1})=0}} \prod_{i=0}^{n-1} \left(\frac{1}{2} + (-1)^{x_i \varepsilon_i} \right) - \frac{1}{2}. \quad (3)$$

By Lemma 1, we need to find all inputs such that $f = 0$, and so the complexity of computing the bias of f is about $\mathcal{O}(2^n)$.

It is worth mentioning that Liu et al. proposed a new key recovery method called conditional differential-linear (CDL) cryptanalysis. CDL cryptanalysis imposes some conditions on the internal bits of the cipher to make the output bits

**Fig. 2.** Differential Algebraic Transitional Form**Algorithm 1** [26] Estimation of the Differential-Linear Bias**Input:** Linear mask λ_o , the expression $Y^{(r)}$, and the set Q .**Output:** A bias ε .

- 1: Calculate the partial derivative of $Y^{(r)} : Y''^{(r)} \leftarrow D_x Y^{(r)}$;
- 2: Compute the ATF of the parity $e = \lambda_o \cdot Y''^{(r)}$, and set $\varepsilon = \frac{1}{2}$;
- 3: **while** $e \neq 0$ **do**
- 4: Select the isolated variables in the ATF of e , and sum them to e_l ;
- 5: Compute the bias of $e^* = e - e_l$ by $\varepsilon^* = \text{Bias}(e^*)$, and calculate $\varepsilon = 2 \cdot \varepsilon^* \cdot \varepsilon$;
- 6: Substitute the expressions Q into e_l , and update e with this new polynomial;
- 7: **Return** ε .

behave non-randomly, which can be used to identify the correct key. To distinguish it from the traditional DL distinguisher, the distinguisher after adding conditions is referred to as the CDL distinguisher. The approach extends the application of DL cryptanalysis from block ciphers to iterative ciphers of other types, including stream ciphers and authenticated encryption algorithms. We refer the readers to [26] for more details.

3 Improving the Precision of Bias Estimation

In this section, we discuss methods to improve the precision of bias estimation. Specifically, we propose three methods to make the bias of DATF closer to that of $f(X) \oplus f(X \oplus \Delta)$.

Transitional rule. The main assumption of the DATF technique is that all variables in the DATF expression are independent. For the internal state $Y^{(r)} = Y'^{(r)} \oplus xY''^{(r)}$ in the r -th round, it is easy to find that there is no algebraic relation between the expressions of each bit in $Y'^{(r)}$, and so the new variables $\alpha^{(r)}$ are independent and uniformly distributed. However, in the previous rounds, all expressions in $Y''^{(r)}$ are simple, non-uniform, and have some linear relationships among them, which stems from the sparse nature of x at the beginning. We will consider two very common cases when replacing $Y''^{(r)}$ with new variables in the first few rounds.

1. When the number of variables in $Y''^{(r)}[i]$ is small, we do not introduce a new variable to replace it. This is because otherwise, new variables are prone to having common variables, and the correlation between new variables will be strong. Generally speaking, the larger the threshold is, the more accurate the bias estimation is at the expense of longer computation time. After the trade-off between accuracy and time, we choose the threshold to be 4. If necessary, we can choose not to replace the expressions of $Y''^{(r)}[i]$ in the first two rounds.
2. We use a dictionary Q to store each variable substitution as $Q[Y''^{(r)}[i]] = \beta^{(r)}[i]$. We can handle some simple linear relation with Q . Specifically, if $T(Y''^{(r)}[i]) \subseteq T(Y''^{(r)}[j])$, then

$$\beta^{(r)}[j] = \beta^{(r)}[i] + Q(Y''^{(r)}[j] \oplus Y''^{(r)}[i]).$$

The above two rules are extensions of the rules in [26,19]. Although both rules are simple, they handle the most common cases in the computation of DATF. Specifically, we apply these two rules in the first few rounds. Since the DATFs of the internal states in the early rounds are relatively simple and sparse, the impact of these rules on the overall complexity is very small.

Backtracking strategy. In Algorithm 1, Liu et al. divided the DATF expression e into $e^* + e_l$, where e_l is the sum of all isolated variables. By substituting e_l with the expressions in Q , the new expression is taken as e , and the decomposition process continues until no isolated variables remain. Assuming this process is performed t times, e can be decomposed into $e_0^* + e_1^* + \dots + e_t^*$. Then, the bias of e is equal to $2^t \prod_{i=0}^t \text{Bias}(e_i^*)$ according to Line 5 of Algorithm 1.

However, upon revisiting the process, we find that there are many common variables among e_i^* . Since these expressions for e_i^* are not complex, treating them as independent will introduce non-negligible errors to the computation result of the bias. Therefore, we propose to compute the bias after expanding e back to the expression without isolated variables. We replace the latest isolated variable each time, and the whole procedure is described in Algorithm 2. It is worth noting that our approach generates an expression with a number of variables, which may lead to the bias computation becoming more difficult. The bias computation method proposed in Section 4 can tackle this problem well.

Next, we present Example 1 to illustrate Algorithms 1 and 2.

Algorithm 2 Improved Estimation of the Differential-Linear Bias**Input:** Linear mask λ_o , the expression $Y^{(r)}$, and the set Q .**Output:** A bias ε .

- 1: Calculate the partial derivative of $Y^{(r)} : Y''^{(r)} \leftarrow D_x Y^{(r)}$;
- 2: Compute the ATF of the parity $e = \lambda_o \cdot Y''^{(r)}$, and set $\varepsilon = \frac{1}{2}$;
- 3: Initialize the set I to be the set of all isolated variables in the ATF of e ;
- 4: **while** $I \neq \emptyset$ **do**
- 5: Select the latest introduced variable e^* in I ;
- 6: Substitute the expressions Q into e^* , and update e with this new polynomial;
- 7: Update I with the new expression of e ;
- 8: Compute the bias $\varepsilon = \text{Bias}(e)$;
- 9: **Return** ε .

Example 1. Let x_0, x_1, x_2, x_3 be four initial variables and y_0, y_1 be two transitional variables such that $y_0 = x_0x_1 \oplus x_2$ and $y_1 = x_1x_2 \oplus y_0$. Set $z = x_2x_3 \oplus y_1$. We would like to estimate the bias of z . On one hand, the computation process of Algorithm 1 could be briefly described by

$$\begin{aligned} \text{Bias}(z) &= 2 \cdot \text{Bias}(x_2x_3) \cdot \text{Bias}(y_1) = 2 \cdot \text{Bias}(x_2x_3) \cdot \text{Bias}(x_1x_2 \oplus y_0) \\ &= 4 \cdot \text{Bias}(x_2x_3) \cdot \text{Bias}(x_1x_2) \cdot \text{Bias}(x_0x_1 \oplus x_2) = 0, \end{aligned}$$

where the bias computation is performed after every iteration. On the other hand, according to Algorithm 2, we only compute the bias until there are no isolated transitional variables, whose process could be described by

$$\begin{aligned} \text{Bias}(z) &= \text{Bias}(x_2x_3 \oplus y_1) = \text{Bias}(x_2x_3 \oplus x_1x_2 \oplus y_0) \\ &= \text{Bias}(x_2x_3 \oplus x_1x_2 \oplus x_0x_1 \oplus x_2) = 2^{-3}. \end{aligned}$$

Partitioning technique. The partitioning technique [4] was first proposed to improve the linear cryptanalysis of ARX ciphers. Subsequently, in [23,19], the technique was introduced into DL cryptanalysis to estimate the differential-linear bias. The basic idea is to partition the whole input space into several smaller, equal-sized subsets, including public and secret variables. We introduce the method for estimating the bias of DATF. This method simplifies the ATF expressions and clarifies the algebraic structure that can represent the bias more effectively. When estimating the differential-linear bias, we identify t variables (or the sum of variables) that frequently appear in the differences of states in the first round and subsequently partition the whole space into 2^t subsets based on these variables. The bias of the distinguisher is then calculated as the average of the biases of these subsets. We provide Example 2 to illustrate the partitioning technique.

Example 2. Let $z = gf_1 \oplus (g \oplus 1)f_2 \oplus f_3$, where $g = g(x_0, x_1, x_2)$, $f_1 = f_1(x_3, x_4, x_5)$, $f_2 = f_2(x_6, x_7, x_8)$, $f_3 = f_3(x_3, x_6, x_9)$. According to the partitioning technique, the entire space can be partitioned into two subspaces according to the value of

g. Consequently, the bias of z adheres to the following relation

$$\begin{aligned}\text{bias}(z) &= \Pr(g = 0) \cdot \text{bias}(z \mid g = 0) + \Pr(g = 1) \cdot \text{bias}(z \mid g = 1) \\ &= \Pr(g = 0) \cdot \text{bias}(f_2 \oplus f_3) + \Pr(g = 1) \cdot \text{bias}(f_1 \oplus f_3).\end{aligned}$$

In this case, the complexity of bias estimation is reduced from 2^{10} to 2^6 . Moreover, the expression under estimation becomes structurally simpler, which in turn ensures that the derived bias is closer to the real value. Notably, due to the feature of iterative ciphers, the output differential function often takes a form similar to z , where g typically corresponds to the differential function of the initial few rounds.

4 A General Bias Computation Method

Given the DATF expression f and assuming that all the variables in f are independent, computing the bias of f is essential for enhancing the efficiency of the DATF technique. In this section, we present a lemma and then provide an efficient method for computing the bias of a Boolean function based on this lemma. The new computation method significantly reduces the computation time and expands the range of applications; it can handle expressions with more variables and more complex structures. It is worth noting that the method described in this section can be used to compute the exact bias of a Boolean function rather than merely estimating it.

Lemma 2. *Given a Boolean function f , if there exists a variable v such that $f|_{v=0}$ is a balanced function, then $\text{bias}(f) = \frac{1}{2}\text{bias}(f|_{v=1})$. Similarly, if there exists a variable v such that $f|_{v=1}$ is a balanced function, then $\text{bias}(f) = \frac{1}{2}\text{bias}(f|_{v=0})$.*

Proof. The probability that $f = 0$ can be computed by

$$\begin{aligned}\Pr(f = 0) &= \Pr(v = 0) \cdot \Pr(f|_{v=0} = 0) + \Pr(v = 1) \cdot \Pr(f|_{v=1} = 0) \\ &= \frac{1}{2} \cdot \left(\frac{1}{2} + \text{bias}(f|_{v=0})\right) + \frac{1}{2} \cdot \left(\frac{1}{2} + \text{bias}(f|_{v=1})\right) \\ &= \frac{1}{2} + \frac{1}{2} \cdot (\text{bias}(f|_{v=0}) + \text{bias}(f|_{v=1})).\end{aligned}$$

Therefore, $\text{bias}(f) = \frac{1}{2} \cdot (\text{bias}(f|_{v=0}) + \text{bias}(f|_{v=1}))$.

Lemma 2 is straightforward, yet it plays a significant role in reducing computational complexity. Given a Boolean function with n variables, if we can identify t variables that satisfy the conditions of Lemma 2, we can reduce the complexity from 2^n to 2^{n-t} when computing the bias. Since fixing one variable may lead to the reduction of several others, the overall complexity is generally below 2^{n-t} . Based on Lemma 2, we preprocess f before computing the bias, that is, we simplify f before computing the bias. The whole procedure is described in Algorithm 3, which consists of a total of four procedures.

- **Bias**(f): Calculate the bias of a Boolean function f .
- **Preprocess**(f): Identify all the variables in f that can satisfy the conditions of Lemma 2 and simplify f based on these variables.
- **RoughBias**(f): Quickly detect whether f is biased. After separating f as a sum of several polynomials with no common variables, the bias is computed only for those with a very small number of variables. If the answer is “0”, then there is no bias.
- **Separate**(f): Separate the Boolean expression into a sum of m polynomials that have no common variables.

Algorithm 3 is an extension of the method proposed by Liu et al. in [26], with an additional preprocessing stage. In the original method, for a polynomial that cannot be separated, if the number of involved variables exceeds 20, some variables are guessed until the variables in all polynomials that cannot be separated is reduced to fewer than 20. Although their method can reduce the number of variables, the total number of variables that need to be considered remains unchanged, which means the computational complexity has not decreased.

The complexity of Algorithm 3. Our method performs multiple preprocessing steps on the function f , significantly reducing the number of variables. Since the extent of complexity reduction achieved by the program **Separate**() is unknown, we do not include this reduction in our complexity analysis. Therefore, the actual complexity will be lower than the value predicted by our theoretical analysis.

Given a Boolean function f with n variables, if the first preprocessing step identifies c variables, then a simpler Boolean function f' with $n - c$ variables will be obtained. According to Line 12 of Algorithm 3, the biases of $f_0 \triangleq f'|_{v_0=0}$ and $f_1 \triangleq f'|_{v_0=1}$ need to be computed separately. Then we repeat this process for f_0 and f_1 , until all the Boolean functions f_i obtained have a small number of variables, i.e., they all satisfy Line 5 of Algorithm 3. The diagram of the entire process is shown in Fig. 3. The dotted lines represent the preprocessing, while the solid lines represent the assignment of values to the functions. The three types of Boolean functions that may occur during the computation process are described in Fig. 3.

- For f'_{01} , the number of variables is less than the pre-selected value m specified in Line 5 of Algorithm 3. The bias can be directly calculated using the Hamming weight.
- For f_{10} , during the preprocessing stage, it was found that there exists a variable v such that both $f_{10}|_{v=0}$ and $f_{10}|_{v=1}$ are balanced. Therefore, f_{10} is balanced and can be pruned directly.
- For f'_{00} and f'_{11} , procedure **Bias**() need to be continued to execute.

Based on the above analysis, the computational complexity $\mathcal{T}$ is the sum of the complexities of the final nodes of all branches in Fig. 3. Suppose the branch with the highest complexity calls procedure **Bias**() t times, and the number of

Algorithm 3 A Bias Computation Method

```

1: procedure Bias( $f$ )
2:    $(f_1, f_2, \dots, f_m) \leftarrow \text{Separate}(f)$ ;
3:    $\varepsilon \leftarrow \frac{1}{2}$ ;
4:   for  $i$  from 1 to  $m$  do
5:     if the number of variables in  $f_i$  is small then
6:       Compute the bias  $\varepsilon_i$  of  $f_i$  according to its Hamming weight;
7:     else
8:        $(f'_i, c) \leftarrow \text{Preprocess}(f_i)$ ;
9:       if  $c = -1$  then
10:        return 0;
11:       Select a variable  $v$  minimizing the maximum cardinality of the variable
12:       sets of the polynomials in  $\text{Separate}(f'_i|_{v=0})$  and  $\text{Separate}(f'_i|_{v=1})$ ;
13:       Compute the bias of  $f'_i$  by  $\varepsilon'_i \leftarrow \frac{1}{2}\text{Bias}(f'_i|_{v=0}) + \frac{1}{2}\text{Bias}(f'_i|_{v=1})$ ;
14:        $\varepsilon_i \leftarrow \frac{1}{2^c} \cdot \varepsilon'_i$ ;
15:        $\varepsilon \leftarrow 2 \cdot \varepsilon \cdot \varepsilon_i$ ;
16:       if  $\varepsilon = 0$  then
17:        break
18:   return  $\varepsilon$ .
   /* The procedure for preprocessing the Boolean polynomial  $f$  */
19: procedure Preprocess( $f$ )
20:    $c \leftarrow 0$ ,  $flag \leftarrow 1$ ;
21:   while  $flag = 1$  do
22:      $flag \leftarrow 0$ ;
23:     for all  $v$  in  $\text{var}(f)$  do
24:        $b_0 \leftarrow \text{RoughBias}(f|_{v=0})$ ,  $b_1 \leftarrow \text{RoughBias}(f|_{v=1})$ 
25:       if  $b_0 = 0$  and  $b_1 = 0$  then
26:        return  $(f, -1)$ .
27:       if  $b_0 = 0$  and  $b_1 = 1$  then
28:         $c \leftarrow c + 1$ ,  $f \leftarrow f|_{v=1}$ ,  $flag \leftarrow 1$ ;
29:        break
30:       if  $b_0 = 1$  and  $b_1 = 0$  then
31:         $c \leftarrow c + 1$ ,  $f \leftarrow f|_{v=0}$ ,  $flag \leftarrow 1$ ;
32:        break
33:   return  $(f, c)$ .
   /* The procedure for quickly and roughly detecting whether the bias is 0 */
34: procedure RoughBias( $f$ )
35:    $(f_1, f_2, \dots, f_m) \leftarrow \text{Separate}(f)$ ;
36:   for  $i$  from 1 to  $m$  do
37:     if the number of variables in  $f_i$  is very small then
38:       Compute the bias  $\varepsilon_i$  of  $f_i$  according to its Hamming weight;
39:       if  $\varepsilon_i = 0$  then
40:        return 0;
41:   return 1.
   /* The procedure for separating the Boolean polynomial  $f$  */
42: procedure Separate( $f$ )
43:   Separate the Boolean polynomial  $f$  into a sum of  $m$  polynomials  $f_i$  whose
44:   variable sets are mutually disjoint;
45:   return  $(f_1, f_2, \dots, f_m)$ .

```

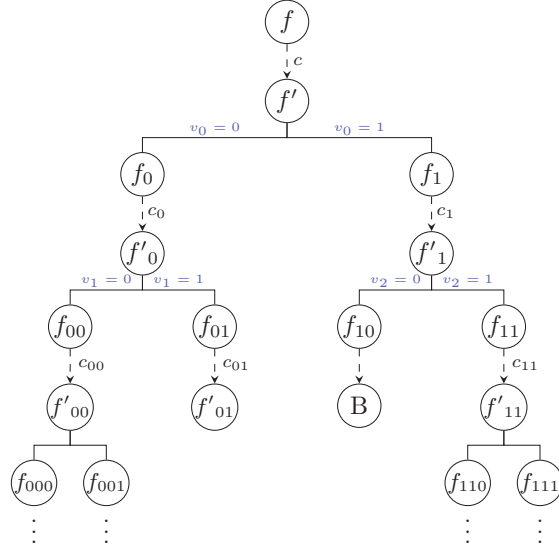


Fig. 3. The process of computing the bias of f .

variables reduced in each preprocessing is $c'_0, c'_1, \dots, c'_t$, respectively. Then the complexity corresponding to this branch is $2^{n-t-\sum_{i=0}^t c'_i}$. Therefore, we have

$$\mathcal{T} < 2^t \times 2^{n-t-\sum_{i=0}^t c'_i} = 2^{n-\sum_{i=0}^t c'_i}.$$

Taking into account the reduction in complexity by the procedure **Separate()**, this actual complexity will be significantly less than $2^{n-\sum_{i=0}^t c'_i}$. The complexity indicates that choosing a relatively small value for the pre-selected parameter m will result in a greater reduction in the number of variables. In the experiments in Section 6, we chose $m = 10$. Furthermore, we observed that most calls to **Preprocess()** identified several variables, which may be attributed to the specific properties of the S-boxes in cryptographic algorithms. In the experiment, the bias computation method can dramatically reduce the computation time by tens or even hundreds of times. Notably, the greater the number of variables in the Boolean expression and the more complex the structure, the more obvious the improvement will be.

Algorithm 3 describes the procedure for computing the bias of a Boolean function. However, the variables involved in DATF are not uniform, i.e., each variable has its own probability distribution. Therefore, when computing the bias for DATF, we need to additionally return the probability p' corresponding to the preprocessed f' in **Preprocess()**, and then modify Line 13 of Algorithm 3 to

$$\varepsilon_i \leftarrow p' \cdot \varepsilon'_i.$$

5 An Effective Method for Searching for Good DL Distinguishers

To search for good DL distinguishers, a common yet simple approach is to explore both the differential positions of the input and the linear positions of the output. However, this method can result in significant computational complexity. Therefore, it is important to propose a method that can quickly evaluate whether a differential-linear pair exhibits a good bias.

Before describing our method, we present the following lemma, which follows directly from the piling-up lemma.

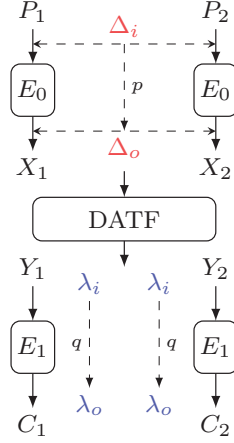
Lemma 3. *Given the DATF expression f and assuming that all variables of f are independent, if there exists an isolated variable in f with zero bias, then the bias of f is also 0. Specifically, for an isolated variable e in f , we have $\text{Bias}(f) \leq \text{Bias}(e)$.*

Searching for DL distinguishers via the DATF technique. According to Lemma 3, when $\text{Bias}(e)$ is very small, we can skip computing the bias of f and discard it. We propose a method that efficiently eliminates small biases. Given a cipher and a threshold ε_b used to determine whether it has a favorable bias, our method is outlined in the following procedure.

1. Given the difference, apply the DATF technique to derive the difference expressions $\beta^{(i)} (i = 1, \dots, r)$ for all internal states, see Fig. 2.
2. Obtain the biases of $\beta^{(i)} (i = 1, \dots, r)$ by iteratively using Lemma 1.
3. For the potential linear mask λ , perform the following steps:
 - (a) Compute the DATF expression $f = \lambda \cdot Y''^{(r)}$ and identify the isolated variables $e_0, e_1, \dots, e_t$.
 - (b) Compute $\varepsilon' = 2^t \cdot \prod_{i=0}^t \varepsilon'_i$, where $\text{Bias}(e_i) = \varepsilon'_i$ are obtained in Step 2.
 - (c) If $\varepsilon' > \varepsilon_b$, calculate the bias of f by Algorithm 2; otherwise, proceed to test the next linear mask.
4. Record all linear masks that satisfy $\text{Bias}(f) > \varepsilon_b$.

Upon analyzing the steps above, it is clear that the first and second steps need to be computed only once for a given difference. This represents a significant advantage of the DATF technique. In contrast, the multi-table method described in [32,17] requires repeating the entire process whenever the difference or linear mask changes. The time complexity of Step 3 is dominated by the exact computation of biases in Step 3(c). Consequently, our method can identify good distinguishers with high efficiency.

Construction of DL distinguishers by combining three parts. The current primary framework for constructing differential-linear distinguishers involves dividing the cipher into three parts: the differential part E_0 , the middle

**Fig. 4.** The three-part framework based on DATF

connection part E_m , and the linear part E_1 . The DATF technique can evaluate the differential-linear bias of E_m , thereby connecting differential and linear trails reasonably, as illustrated in Fig. 4. We refer to the output difference Δ_o of E_0 and the input mask λ_i of E_1 as the difference and linear mask of DATF, respectively. Then, the bias of the entire DL distinguisher is

$$\varepsilon = 4 \cdot p \cdot \varepsilon_{\text{DATF}(\Delta_o \rightarrow \lambda_i)} \cdot q^2. \quad (4)$$

There are two common methods for constructing DL distinguishers by combining the three parts.

- Search for many good differential-linear pairs (Δ_o, λ_i) for E_m and then expand them by finding the high-probability differentials of E_0 and the high-bias linear approximations of E_1 .
- Search for many high-probability differentials $\Delta_i \xrightarrow{p} \Delta_o$ and high-bias linear approximations $\lambda_i \xrightarrow{q} \lambda_o$ for E_0 and E_1 , respectively. Then, enumerate all combinations (Δ_o, λ_i) and estimate the differential-linear bias of E_m .

We adopt the second approach herein. A key advantage of this method is that all differential/linear trails with biases exceeding a predefined threshold can be obtained by solving a single Mixed Integer Linear Programming (MILP) model. Subsequently, leveraging Lemma 3, we efficiently screen high-bias differential-linear pairs from all (Δ_o, λ_i) combinations. Finally, according to formula (4), we select the optimal DL distinguisher that integrates the three components.

In contrast, although the first approach can initially yield good (Δ_o, λ_i) pairs for E_m , it has two critical limitations. First, searching for matching differential and linear trails requires repeatedly solving the MILP model, which significantly reduces search efficiency. Second, for a given (Δ_o, λ_i) pair, it is difficult to find both a high-probability differential trail (for E_0) and a high-bias linear approximation (for E_1), as such trails or approximations are rare in practice.

6 Applications

6.1 Applications to Ascon

Ascon [13], designed by Dobraunig et al., is a family of AEAD and hash algorithms. It was selected as the new standard for lightweight authenticated encryption and hashing in the NIST Lightweight Cryptography competition (2019 - 2023). More recently, the Ascon family has been specified in NIST SP 800-232 [30]. The original submissions to the LWC project included two versions, Ascon-128 and Ascon-128a, as described in [13], along with the later NIST standardized version Ascon-AEAD128 as described in [30].

A brief description of the Ascon initialization. For the Ascon initialization, the 320-bit internal state S is divided into five 64-bit words: X_0 , X_1 , X_2 , X_3 , and X_4 . The state at the output of the r -th round is denoted by $X_0^r \| X_1^r \| X_2^r \| X_3^r \| X_4^r$, while $Y_0^r \| Y_1^r \| Y_2^r \| Y_3^r \| Y_4^r$ represents the state after the p_S layer. We use $X_i^r[j]$ (resp. $Y_i^r[j]$) to denote the j -th bit (starting from left) of X_i^r (resp. Y_i^r). The 128-bit key $(k_0, k_1, \dots, k_{127})$ and 128-bit nonce $(v_0, v_1, \dots, v_{127})$ are set to 256 binary variables, and the nonce is set to a constant defined by the cipher. The core permutation p of Ascon is based on a substitution permutation network (SPN). It consists of three operations: the round constant addition p_C , the substitution layer p_S , and the linear layer p_L , as $p = p_L \circ p_S \circ p_C$. Since the linear layer is invertible for a given output, we do not consider the linear layer of the last round.

To obtain a simple ATF expression, the DATF technique performs variable substitution to the state after the round function. For Ascon, the authors of [26,19] chose to perform the ATF transformation at a specific time rather than at the end of each round function. First, they divide the S-box into two parts p_{S_L} and p_{S_N} , where p_{S_L} is a linear transformation. Then, the r -round Ascon is represented as

$$E = p_{S_N} \circ (p_{S_L} \circ p_C \circ p_L \circ p_{S_N})^{r-1} \circ p_{S_L} \circ p_C.$$

Clearly, after all linear operations are completed, substituting variables results in simpler ATFs with a bias closer to the true value. So, like [26,19], we also perform ATF transformation after each p_{S_L} .

The adversary can only access the first word of the output state for Ascon-128 and the first two words for Ascon-128a and Ascon-AEAD128. In addition, for these three versions, there are data block limitations for a given key: Ascon-128 and Ascon-128a have a limit of 2^{64} data blocks, while Ascon-AEAD128 has a limit of 2^{50} data blocks. In the following text, if we do not emphasize the version, it indicates that the attack is effective for all three versions.

Key-recovery attack on 4-round Ascon. We first apply the enhanced DATF technique to the 4-round Ascon by selecting the input difference $\Delta(0) = \{v_0, v_{64}\}$ and setting $v_0 = v_{64}$. Based on the algebraic transitional representation of the

Table 3. The bias of some output bits of 4-round Ascon

Output bit	Key-bit pair (k_0, k_{64})			
	(0,0)	(0,1)	(1,0)	(1,1)
$Y_0^4[21]$	2^{-1}	-	-	$2^{-2.415}$
$Y_0^4[30]$		$2^{-1.415}$	0	-
$Y_0^4[50]$	2^{-1}	-	-	2^{-1}

DATF technique, we find some special algebraic structures on the output bits, which can directly yield the corresponding bias. These bits are as follows,

$$Y_0^4[21] = k_0 \cdot p_0 + k_{64} \cdot p_1, \quad Y_0^4[50] = (k_0 + k_{64}) \cdot p_2,$$

where p_0, p_1, p_2 are different expressions. Then, based on these two bits, we can distinguish between $(k_0, k_{64}) = (0, 0)$ and $(k_0, k_{64}) = (1, 1)$. Next, we employ the enhanced DATF technique to estimate the bias for the remaining two scenarios, with results summarized in Table 3. With the help of Table 3, the attacker can recover two key bits by matching the expected bias with the calculated bias. Due to the rotation-invariant property of Ascon, the same method can be used to recover the other key bits. Using the formula (1), the success probability of distinguishing two correct keys with 2^5 samples is approximately 99%. This indicates that the time complexity and data complexity of the 4-round key recovery attack are both 2^{11} .

Key-recovery attack on 5-round Ascon. We revisit the CDL distinguisher of 5-round Ascon in [26] with the enhanced DATF technique. Consistent with [26], for a given difference $\Delta(0)$ and the default setting $v_0 = v_{64}$, we impose the conditions in the first two rounds and then estimate the bias of the 5-round CDL distinguisher. For the output mask λ_o in $Y_0^5[27]$, we obtain the estimated bias 2^{-5} , which is closer to the exact bias $2^{-4.54}$ than the estimated value $2^{-5.5}$ by Liu et al. This demonstrates the effectiveness of our improvements.

Furthermore, we also find a new CDL distinguisher with a larger bias. For the output mask λ_o in $Y_0^5[47]$, we estimate the bias to be 2^{-3} , while the experimental bias is $2^{-2.99}$. The 6 equations that need to be satisfied by this CDL distinguisher are as follows:

$$k_0 = 0,$$

$$k_{64} = 0,$$

$$v_2 = k_2 + k_9 + k_{12}v_{76} + k_{12} + k_{19}v_{83} + k_{42}v_{106} + k_{42} + k_{66} + k_{73} + k_{83} \\ + v_{12} + v_{42} + v_{66} + v_{76} + v_{106},$$

$$v_{67} = k_3k_{67} + k_3v_3 + k_3 + k_{25}k_{89} + k_{25}v_{25} + k_{25} + k_{67}v_3 + k_{67} + k_{89}v_{25} \\ + k_{89} + v_3 + v_{25} + v_{89} + 1,$$

$$v_{86} = k_{19}k_{83} + k_{19}v_{19} + k_{19} + k_{22}k_{86} + k_{22}v_{22} + k_{22} + k_{44}k_{108} + k_{44}v_{44} + k_{44} + k_{83}v_{19} \\ + k_{83} + k_{86}v_{22} + k_{86} + k_{108}v_{44} + k_{108} + v_{19} + v_{22} + v_{44} + v_{83} + v_{108},$$

$$v_{95} = k_{28}k_{92} + k_{28}v_{28} + k_{28} + k_{31}k_{95} + k_{31}v_{31} + k_{31} + k_{53}k_{117} + k_{53}v_{53} + k_{53} + k_{92}v_{28} \\ + k_{92} + k_{95}v_{31} + k_{95} + k_{117}v_{53} + k_{117} + v_{28} + v_{31} + v_{53} + v_{92} + v_{117}.$$

Table 4. The relevant normalized entries of the DLCT in the S-box no.57

$\overline{\text{DLCT}}_S(0, 16) = 2^{-1}$	$\overline{\text{DLCT}}_S(1, 16) = 0$	$\overline{\text{DLCT}}_S(2, 16) = -2^{-1}$	$\overline{\text{DLCT}}_S(3, 16) = 0$
$\overline{\text{DLCT}}_S(4, 16) = 0$	$\overline{\text{DLCT}}_S(5, 16) = -2^{-1}$	$\overline{\text{DLCT}}_S(6, 16) = 0$	$\overline{\text{DLCT}}_S(7, 16) = 2^{-1}$
$\overline{\text{DLCT}}_S(16, 16) = 0$	$\overline{\text{DLCT}}_S(17, 16) = -2^{-1}$	$\overline{\text{DLCT}}_S(18, 16) = 0$	$\overline{\text{DLCT}}_S(19, 16) = 2^{-1}$
$\overline{\text{DLCT}}_S(20, 16) = 2^{-1}$	$\overline{\text{DLCT}}_S(21, 16) = 0$	$\overline{\text{DLCT}}_S(22, 16) = -2^{-1}$	$\overline{\text{DLCT}}_S(23, 16) = 0$

Our experiments show that the bias is significantly smaller than 2^{-4} when one or more than one of the above equations are not satisfied. Fixing the nonce variables appearing on the right side of the above equations as constant 0, $2^{9.5+4}$ samples with $v_2, v_{67}, v_{86}, v_{95}$ running over possible values are sufficient to distinguish $k_0 = k_{64} = 0$ from the other cases. Similar to the method in [26], we can recover 19 extra bits of key information for the case $k_0 = k_{64} = 0$. Taking the equation corresponding to v_2 as an example, when v_{76}, v_{83} and v_{106} are respectively taken as 1, we can obtain the values of k_{12}, k_{19} and k_{42} with $3 \times 2^{9.5}$ samples.

We can detect 64-bit positions of the word, and on average, 16-bit positions satisfy $k_i = k_{i+64} = 0$. With only 9 such cases, we could get $7 \times 21 = 147$ linear equations to recover all the keys. Therefore, the data complexity of the attack is on average $64 \times 2^{9.5+4} + 7 \times 19 \times 2^{9.5} \approx 2^{19.68}$ bits, and the expected time complexity is about $2^{19.68}$, for most of the keys.

Comments on the distinguisher of 6-round Ascon in [32]. At CRYPTO 2024, the authors of [32] presented a 6-round DL distinguisher for Ascon. They found a 5-round truncated difference, where the 57th S-box output truncated difference is $*0***$. Then, assuming that each input difference probability of the 57th S-box is 2^{-4} , combined with formula (2), the bias calculation formula is as

$$\varepsilon = 4 \cdot p \cdot 2^{-4} \left(\sum_{\Delta=*0*** \in \mathbb{F}_2^4} \overline{\text{DLCT}}_S(\Delta, 16) \right) \cdot q^2.$$

However, Peng et al. ignored the fact that positive and negative biases can cancel each other, resulting in a decrease in the overall bias. We provide relevant normalized entries of the DLCT of S-box in Table 4. Based on Table 4, we easily obtain that the bias is 0, that is, this distinguisher is invalid. Furthermore, we utilize CUDA for experimental verification. We ensure one round of deterministic propagation and detect no reasonable bias even using 2^{45} samples.

Key-recovery attack on 6-round Ascon-AEAD128 and Ascon-128a. Liu et al. and Hu et al. have both noted that constructing a 6-round distinguisher is extremely challenging, as described in [26] and [19]. However, using our method, we obtain a 6-round CDL distinguisher and present a key recovery attack on 6-round Ascon-AEAD128 and Ascon-128a. This is the first DL cryptanalysis against 6-round Ascon. We first introduce a 5-round CDL distinguisher for Ascon and then extend it by one round to construct the 6-round CDL distinguisher. Based on previous analysis, that is, with the same difference and conditions, we

Table 5. Comparison of bias of 9-round DL distinguishers

Start round	r_0	r_m	r_1	Experimental value	Theoretical bias		
					[32]	[17]	Sect. 6.2
1	2	2	5	$2^{-52.00}$	2^{-52}		2^{-52}
1*	2	2	5	$2^{-56.98}$	2^{-52}		2^{-57}
2	1	3	5	$2^{-55.33}$	$2^{-55.33}$		$2^{-55.26}$
1	2	3	4	$2^{-51.89}$		$2^{-51.95}$	$2^{-50.96}$
2	2	3	4	$2^{-55.42}$		$2^{-55.43}$	$2^{-55.40}$

* The result was published at Crypto 2024 in [32] and updated by the authors almost one year later.

search for a CDL distinguisher for five rounds (with an additional linear layer compared to before).

We estimate that the bias of X_4^5 [47] is -2^{-12} , while the bias of the remaining four positions of the S-box is 0. This is also verified by our experiments. Next, we expanded an S-box using linear trails: for an S-box with input mask 00001 and output mask 11000, the linear approximation bias is 2^{-3} . Using the formula (4), the bias of this differential-linear distinguisher is calculated as $4 \cdot p \cdot 2^{-12} \cdot 2^{-6} = 2^{-16}$, which is close to the experimental bias $\varepsilon = 2^{-15.91}$ we later obtained using 2^{43} samples.

Similar to the key recovery method for 5-round Ascon, we first identify positions where $k_i = k_{i+64} = 0$, then recover additional key information. For most keys, the average data complexity of the attack is $64 \times 2^{35.4+4} + 7 \times 19 \times 2^{35.4} \approx 2^{45.58}$, with an expected time complexity of approximately $2^{45.58}$. Using the formula (1), the success probability of the attack is more than 99%.

6.2 Applications to Serpent

Serpent [3], designed by Anderson, Biham, and Knudsen, is a finalist in the AES competition. It has a 128-bit block size and supports a key size of 128, 192, or 256 bits. Serpent is a 32-round SP-Network consisting of alternating layers of key mixing, S boxes, and linear transformation. It uses eight 4-bit to 4-bit S-boxes, denoted by $S_0, S_1, \dots, S_7$, respectively. Each round function uses a single S-box applied 32 times in parallel. In our description, we adopt the notations of [3] in the bitsliced version, i.e., the internal state is represented as four 32-bit words.

Revisiting the 9-round distinguisher in [32,17]. To verify the effect of our approach, we revisit the distinguishers for 9-round Serpent given in [32,17]. For comparison, we list the experimental bias and our estimated bias in Table 5. All the experimental biases in this section are that only the E_m part was experimented. For more details of the distinguishers, please refer to Table 10.

New 9-round distinguisher. In this part, we apply the search method described in Section 5 to search for better distinguishers. Based on a simple anal-

Table 6. A 10-round DL distinguisher

offset = 1, $r_0 = 2, r_m = 3, r_1 = 5, p = 2^{-12}, r = 2^{-10.95}, q = 2^{-23}$			
Δ_i	1d09000e000000000000000000000000	Δ_o	00400000000000000000000000000000100002
λ_i	00000000000000000000000000000000	λ_o	000b0000b000030000b0200e00000010

ysis of Serpent, we prefer to search for 9-round DL distinguishers with $r_0 = 1$, $r_m = 3$, and $r_1 = 5$. We first identify some differential characteristics with high probability and linear approximations with high bias, then use DATF to connect them. Table 10 summarizes our results for the 9-round Serpent. We present the best 9-round distinguisher so far, thus demonstrating the effectiveness of our approach. It is worth noting that for the 9-round DL distinguisher, we obtain many more biases than those listed in the table.

Better key recovery attack on 11-round Serpent. We apply the method to 10-round Serpent but do not find any approximation with a bias greater than 2^{-64} . In particular, we find a 10-round distinguisher with a theoretical bias of $2^{-66.95}$ and use it to give a key recovery attack on 11-round Serpent. Surprisingly, the experimental bias also matched $2^{-66.95}$, that the estimated bias of $\Delta_o \xrightarrow{R^3} \lambda_i$ aligns well with the observed experimental bias. The details of the 10-round DL distinguisher are provided in Table 6. We analyze the differential part of 10-round distinguisher, schematically shown in Fig. 5, attempting to add some conditions to improve the bias of the distinguisher.

In the attack, we use an input difference with 4 active S-boxes 0, 1, 3, 7:

$$\Delta_i = \{0, 1, 3, 39, 65, 71, 97, 99, 103\},$$

and impose the following 4 equations:

$$\begin{aligned} v_1 &= v_{97} \oplus k_1 \oplus k_{97}; & v_{33} &= k_{33}; \\ v_3 &= v_{99} \oplus k_3 \oplus k_{99}; & v_{35} &= k_{35}, \end{aligned}$$

where v_i and k_i respectively denote bit i of plain-text input to round 1 and the 128-bit subkey K_1 of round 1. When these four conditions are satisfied, the difference through S-boxes 1 and 3 will become a deterministic difference, see the first two fat arrows in Fig. 5.

After guessing the values of $k_1 \oplus k_{97}, k_3 \oplus k_{99}, k_{33}, k_{35}$, the attack process is consistent with [6,26] as described below.

1. Select $N = 2^{120.8}$ plaintexts, consisting of $2^{112.8}$ structures, each is chosen by selecting:
 - (a) Any plaintext P_0 .
 - (b) The plaintexts $P_1, \dots, P_{2^8-1}$ which differ from P_0 by all the $2^8 - 1$ possible (non-empty) subsets of the eight bits which enter the active S-boxes 0, 7 in round 1. In particular, v_1, v_3, v_{33}, v_{35} are determined according to the guessed key expression.

2. Request the ciphertexts of these plaintext structures (encrypted under the unknown key K).
3. For each value of the 8 bits of K_1 entering these 2 S-boxes:
 - (a) Initialize an array of 2^8 counters to zeroes.
 - (b) Partially encrypt each plaintext through the 2 active S-boxes in round 1 and find the pairs that satisfy the difference Δ_i before round 2.
 - (c) Given those $2^{119.8}$ pairs, perform for each ciphertext pair: count over all pairs how many times each of the 2^{56} possibilities of the 56 bits entering the 7 active S-boxes in round 11 occurs.
 - (d) For each guess of the subkey entering these S-boxes, find how many pairs agree on the output subset parity, and how many disagree.
 - (e) The highest entry in the array should correspond to the 28 bits of K_{11} entering the 7 active S-boxes in round 11.
4. Each trial of the key gives us $8 + 28 = 36$ bits of the subkeys (8 bits in round 1 and 28 bits in round 11), along with a measure for correctness.
5. The rest of the key bits are then recovered by auxiliary techniques.

The above attack steps need to be repeated 2^4 times because four key expressions are being guessed. The data complexity of the attack is $2^{124.8}$ chosen plaintexts, the time complexity is $2^4 \cdot 2^{120.8} \cdot 2^8 \cdot \frac{2}{352} = 2^{125.34}$ encryptions, and the memory complexity is 2^{60} bytes for the 11-round attack. When all keys in the process are guessed correctly, the bias of the distinguisher increases to $2^{-57.95}$. According to formula (1), the success probability of the attack is expected to be about 85%. The attack based on our distinguisher is the best known attack on 11-round Serpent in terms of both time and data complexity.

6.3 Applications to Xoodyak

Xoodyak [11] is a cryptographic primitive designed for hashing, authenticated encryption, and MAC computation. It is one of the ten finalists in the NIST LWC competition. The internal state consists of 384 bits, represented as a $4 \times 3 \times 32$ cube, with the entire state denoted as $S[x][y][z]$. When y is fixed, the 128 bits of $S[\cdot][y][\cdot]$ are called a plane. Xoodyak uses two modes: hash mode and keyed mode. In this section, we focus on the keyed mode and its initialization phase: the first plane is initialized with a 128-bit key, and the remaining two planes are initialized with a 256-bit nonce. Subsequently, the round function is performed on the initialized state, and the first 192 bits are visible and XORed with the first block of the plaintext.

The round function consists of five operations as $\rho_{east} \circ \chi \circ \iota \circ \rho_{west} \circ \theta$, with only χ being nonlinear. Similar to Ascon, we rearrange the round function to better accommodate the DATF technique. The R -round Xoodyak is expressed as

$$E = \rho_{east} \circ \chi \circ (\iota \circ \rho_{west} \circ \theta \circ \rho_{east} \circ \chi)^{R-1} \circ \iota \circ \rho_{west} \circ \theta.$$

After all linear operations are completed, substituting variables results in simpler ATFs with a bias closer to the actual value. Therefore, we perform the ATF transformation after each ι .

Table 7. The (higher-order) differential-linear bias of round-reduced Xoodyak

Rounds	Order	Experimental value	Theoretical estimate	
			HATF [19]	Sect.6.4
4	1 st	$2^{-9.7}$	$2^{-9.67}$	$2^{-9.68}$
	2 nd	$2^{-5.72}$	$2^{-5.72}$	$2^{-5.72}$
	4 th	2^{-1}	2^{-1}	2^{-1}
5	1 st †	$2^{-9.36}$	2^{-10}	$2^{-9.36}$
	2 nd	-	2^{-45}	$2^{-42.88}$
6	2 nd †	-	-	$2^{-48.47}$

† This distinguisher is under the related-key model.

Revisiting the DL and the HDL distinguishers in [19,15]. In [15], Dunkelman and Weizman gave the first DL attacks on 4-round Xoodyak under the single-key model and on 5-round Xoodyak under the related-key model. The two distinguishers used in these attacks are detected by experiments. Subsequently, Hu et al. used the HATF technique to give the theoretical biases for these distinguishers. We revisit these two distinguishers using the enhanced DATF and the results are listed in Table 7. Our theoretical bias values align closely with the experimental results, confirming the accuracy of our method. In Table 7, we also revisit the HDL distinguishers presented in [19].

2nd-order HDL distinguishers for 6-round Xoodyak under the related-key model. Based on the new framework, we divide the 6-round HDL distinguisher into a 1-round differential trail and a 5-round HDL approximation. First, we describe the search for 5-round HDL approximations. We enumerate the 2nd-order combinations of the three differences

$$(S[i][0][j], S[i][1][j]), (S[i][0][j], S[i][2][j]), (S[i][1][j], S[i][2][j]),$$

where $0 \leq i \leq 3, 0 \leq j \leq 31$. Because of the rotation-invariant property of Xoodyak, when exhausting the 2nd-order difference input, we can fix one difference and exhaust the other difference. Using the enhanced DATF, the best result we found is that the input differences occur in $(S[0][0][0], S[0][1][0])$ and $(S[1][0][4], S[1][1][4])$, the highest bias appears in $S^{(5)}[1][1][2]$ which is $2^{-39.47}$. Next, we add another round of difference trails before these two differences, i.e.,

$$\Delta_0 \xrightarrow{p=2^{-4}} (S[0][0][0], S[0][1][0]), \Delta_1 \xrightarrow{p=2^{-4}} (S[1][0][4], S[1][1][4]),$$

where $\Delta_0 = \theta^{-1}(S[0][0][0], S[3][1][31])$ and $\Delta_1 = \theta^{-1}(S[1][0][4], S[0][1][3])$. Therefore, the entire 2nd-order HDL distinguisher for the 6-round Xoodyak under the related-key model has a bias of $2^{-48.47}$.

Key-recovery attack on 6-round Xoodyak under the related-key model. Now, we apply the new distinguisher to conduct a key recovery attack on the

6-round Xoodyak under the related-key model. We select the 2nd-order difference as (Δ_0, Δ_1) . Eight linear conditions are injected into the first round; after six rounds, the bias of $S^{(6)}[1][1][2]$ is predicted to be $2^{-39.47}$. These eight conditions are detailed in Appendix C. According to formula (1), using 2^{84} samples yields a success probability exceeding 99% for distinguishing eight correct keys. By repeating this process for four additional positions via rotation, we can recover 32 more bits of key information. The remaining key bits can then be obtained through exhaustive search. Therefore, the overall time and data complexity is $5 \times 2^8 \times 2^{84} = 2^{94.32}$.

6.4 Applications to Grain v1

Grain v1 [18], designed by Hell et al., is one of the finalists selected in the eSTREAM hardware profile. It is an NFSR-based stream cipher with a 160-bit internal state. Grain v1 uses an 80-bit secret key $K = (k_0, k_1, \dots, k_{79})$ and a 64-bit initial value $V = (v_0, v_1, \dots, v_{63})$. The cipher consists of two cascaded registers: an NFSR and an LFSR, with keystream bits generated by Boolean functions possessing specific cryptographic properties. In this paper, we discuss round-reduced variants of Grain v1 with r initialization rounds, meaning the cipher outputs keystream after r rounds, and the first keystream bit is z_r .

At ASIACRYPT 2010, Knellwolf et al. [21] proposed conditional differential attacks on NFSR-based cryptosystems. Since its proposal, it has been the most effective key recovery attack method for Grain v1. The idea of conditional differential attacks is to slow the diffusion of differences by controlling the values at certain positions in the nonce. Additionally, specific conditions can be exploited to recover key information. The CDL cryptanalysis proposed by Liu et al. inherits this idea, with the conditional differential attack representing an extreme case of CDL attacks.

Searching for the differences of round-reduced Grain v1. In 2019, Li and Guan [24] proposed a conditional differential attack on 120-round Grain v1, utilizing a conditional differential distinguisher with an experimental bias of $2^{-12.8}$. Later, Liu et al. revisited this analysis and gave an attack on 125-round Grain v1. We now apply our method to these two results. Our experiments reveal that the new variables introduced by the DATF technique for Grain v1 exhibit small biases. For efficiency, we consider these new variables as independent and uniformly distributed.

Using the method described in Section 5, we perform an exhaustive search over all differences involving at most four bits of the initial value. For the 126-round Grain v1, we find an effective conditional differential distinguisher and conduct experimental verification. The input difference that we find is a 2-bit difference, specifically in bits 22 and 47 of the initial value. As a comparison, all results are summarized in Table 8. We remark that distinguishers were also found for the 127- and 128-round Grain v1; however, the number of keys that needed to be guessed was excessive, preventing a valid key recovery attack.

Table 8. The differential-linear bias of round-reduced Grain v1

Rounds	Experimental value	Theoretical estiamte		
		Differential Engine [1]	DATF [26]	Sect.6.4
120	$2^{-12.8}$ [24]	$2^{-18.13}$	$2^{-13.39}$	$2^{-13.105}$
125	$2^{-17.4}$ [26]	$2^{-24.78}$	$2^{-20.77}$	$2^{-17.163}$
126	$2^{-17.4}$	-	-	$2^{-17.163}$

Key recovery attack on 126-round Grain v1. A condition in which a specific nonce bit is fixed to 0 or 1 is called a Type 0 condition, and a condition that depends on both nonce bits and key bits is called a Type 1 condition. By analyzing six ATFs for the 126-round Grain v1, we transform them into 11 Type 0 and 14 Type 1 conditions, see Appendix D. These equations yield a total of 21 independent expressions involving key bits that need to be guessed during the attack. By formula (1), when using 2^{38} plaintext pairs, the success probability of the attack is about 85%. Therefore, the data complexity is $2^{14} \times 2^{39} = 2^{53}$ and the time complexity is $2^{21} \times 2^{39} = 2^{60}$. To the best of our knowledge, this is the best-known analysis of the initialization for Grain v1 in the single key setting.

7 Discussions and Open Problems

Here, we compare the enhanced DATF with existing bias evaluation methods. Notably, evaluating bias requires consideration of both accuracy and computational complexity, two key factors in assessing the practicality of a method.

First, we compare the enhanced DATF with the original DATF proposed by Liu et al. [26]. The original DATF performs variable substitution directly for all expressions, resulting in a complexity of $\mathcal{O}(2^d r t_R)$ when constructing DATF expressions. Here, t_R denotes the complexity of computing the ANFs of the round function R , and d is the algebraic degree of R . By contrast, the enhanced DATF integrates the transitional rules presented in Section 3 to explicitly match the relationships between expressions. This modification adjusts its complexity to $\mathcal{O}(r 2^d t_R + r M n^2 / 2)$, where M represents the maximum number of terms in the ANF of R , and n is the size of the internal state. After adopting our new bias computation method, the time required for bias calculation becomes negligible compared to the time taken to construct DATF expressions. Although the overall computational complexity increases slightly, the enhanced DATF enables the identification of numerous differential-linear pairs that were inaccessible to the original DATF, thereby addressing a key limitation of the original method.

Next, we compare the enhanced DATF with the bias evaluation methods proposed in [17,32]. For the methods in [17,32], the complexity of calculating the bias r is approximately r^{-2} . Importantly, this complexity grows exponentially with the number of active S-boxes, and these methods do not offer a significant speed advantage over traditional experimental approaches. Additionally, for each active S-box, these methods require storing multiple large-scale lookup tables, resulting in exponentially increasing memory complexity. This dual exponential

Table 9. Comparison of complexity and accuracy for bias evaluation methods

Methods	DATF [26]	Enhanced DATF	GDLCT [17]	TDT [32]	Experiment
Complexity	$\mathcal{O}(r2^d t_R)$	$\mathcal{O}(r2^d t_R + rMn^2/2)$	$\mathcal{O}(r^{-2})$		$\mathcal{O}(r^{-2})$
Accuracy	Approximate	Moderately accurate	Highly accurate		Accurate

overhead makes both methods impractical for ciphers with a large number of active S-boxes or large-sized S-boxes. In contrast, the complexity of our method primarily depends on the round function R and is independent of the bias value itself. When combined with our proposed search strategy, it enables efficient searching for high-quality biases and provides clear advantages over experimental computation methods. Notably, our method performs more effectively, efficiently, and accurately on low-degree S-boxes, such as the χ function.

As shown in Table 5, for certain 9-round Serpent distinguishers, the methods proposed in [17,32] provide more accurate bias estimations than ours. Although these methods in [17,32] yield more precise estimations, their computational complexities are nearly the same as that of the experimental method. In contrast, our method achieves near-accurate bias estimations in just 0.5 seconds, demonstrating high efficiency. A key advantage of our method is its ability to efficiently identify potential high-bias distinguishers. Once these candidate distinguishers are identified, their actual biases can be further verified through subsequent experimental tests. In Table 9, we present a straightforward comparison of the complexity and accuracy of the existing methods.

Open Problems and Future Works. Our research raises several open problems as well as some suggestions for future work.

1. In the DL cryptanalysis, the main approach involves dividing the cipher into three parts to search for DL distinguishers. The selection of differential trails, linear trails, and middle differential-linear pairs significantly influences the bias. Although the biases of DL distinguishers can now be effectively evaluated, finding optimal distinguishers remains an interesting problem for future research.
2. Computing the lower bounds on differential-linear distinguishers helps evaluate the security of a cipher against DL cryptanalysis. For example, it is uncertain whether a DL distinguisher with a bias greater than 2^{-64} exists for the 10-round Serpent. Despite making great efforts, we failed to find one.
3. Note that bias estimation is of interest to many cryptanalysis methods, such as differential, linear, and differential-linear cryptanalysis. Therefore, proposing a general and efficient method for computing the bias of a Boolean function is an important problem.

8 Conclusion

In this paper, we enhance the DATF technique in three aspects: achieving higher precision and efficiency in bias estimation, and enabling an effective search for

DL distinguishers. We apply the enhanced DATF to the NIST standard Ascon, the AES finalist Serpent, the NIST LWC finalist Xoodyak, and the eSTREAM finalist Grain v1, and obtain some better results. We believe that the enhanced DATF technique would be a useful tool for the cryptanalysis and design of symmetric-key primitives.

Acknowledgements. The authors would like to thank anonymous reviewers for their helpful comments and suggestions. The work of this paper was supported by the National Natural Science Foundation of China under Grant No. 62372464.

References

1. Banik, S.: Some insights into differential cryptanalysis of grain v1. In: Susilo, W., Mu, Y. (eds.) Information Security and Privacy - 19th Australasian Conference, ACISP 2014, Wollongong, NSW, Australia, July 7-9, 2014. Proceedings. Lecture Notes in Computer Science, vol. 8544, pp. 34-49. Springer (2014). https://doi.org/10.1007/978-3-319-08344-5_3, https://doi.org/10.1007/978-3-319-08344-5_3
2. Bar-On, A., Dunkelman, O., Keller, N., Weizman, A.: DLCT: A new tool for differential-linear cryptanalysis. In: Ishai, Y., Rijmen, V. (eds.) Advances in Cryptology - EUROCRYPT 2019 - 38th Annual International Conference on the Theory and Applications of Cryptographic Techniques, Darmstadt, Germany, May 19-23, 2019, Proceedings, Part I. Lecture Notes in Computer Science, vol. 11476, pp. 313-342. Springer (2019). https://doi.org/10.1007/978-3-030-17653-2_11, https://doi.org/10.1007/978-3-030-17653-2_11
3. Biham, E., Anderson, R.J., Knudsen, L.R.: Serpent: A new block cipher proposal. In: Vaudenay, S. (ed.) Fast Software Encryption, 5th International Workshop, FSE '98, Paris, France, March 23-25, 1998, Proceedings. Lecture Notes in Computer Science, vol. 1372, pp. 222-238. Springer (1998). https://doi.org/10.1007/3-540-69710-1_15, https://doi.org/10.1007/3-540-69710-1_15
4. Biham, E., Carmeli, Y.: An improvement of linear cryptanalysis with addition operations with applications to FEAL-8X. In: Joux, A., Youssef, A.M. (eds.) Selected Areas in Cryptography - SAC 2014 - 21st International Conference, Montreal, QC, Canada, August 14-15, 2014, Revised Selected Papers. Lecture Notes in Computer Science, vol. 8781, pp. 59-76. Springer (2014). https://doi.org/10.1007/978-3-319-13051-4_4, https://doi.org/10.1007/978-3-319-13051-4_4
5. Biham, E., Dunkelman, O., Keller, N.: Enhancing differential-linear cryptanalysis. In: Zheng, Y. (ed.) Advances in Cryptology - ASIACRYPT 2002, 8th International Conference on the Theory and Application of Cryptology and Information Security, Queenstown, New Zealand, December 1-5, 2002, Proceedings. Lecture Notes in Computer Science, vol. 2501, pp. 254-266. Springer (2002). https://doi.org/10.1007/3-540-36178-2_16, https://doi.org/10.1007/3-540-36178-2_16
6. Biham, E., Dunkelman, O., Keller, N.: Differential-linear cryptanalysis of serpent. In: Johansson, T. (ed.) Fast Software Encryption, 10th International Workshop, FSE 2003, Lund, Sweden, February 24-26, 2003, Revised Papers. Lecture Notes in Computer Science, vol. 2887, pp. 9-21. Springer (2003). https://doi.org/10.1007/978-3-540-39887-5_2, https://doi.org/10.1007/978-3-540-39887-5_2

7. Biham, E., Shamir, A.: Differential cryptanalysis of des-like cryptosystems. In: Menezes, A., Vanstone, S.A. (eds.) *Advances in Cryptology - CRYPTO '90*, 10th Annual International Cryptology Conference, Santa Barbara, California, USA, August 11-15, 1990, Proceedings. *Lecture Notes in Computer Science*, vol. 537, pp. 2–21. Springer (1990). https://doi.org/10.1007/3-540-38424-3_1, https://doi.org/10.1007/3-540-38424-3_1
8. Blondeau, C., Leander, G., Nyberg, K.: Differential-linear cryptanalysis revisited. In: Cid, C., Rechberger, C. (eds.) *Fast Software Encryption - 21st International Workshop, FSE 2014, London, UK, March 3-5, 2014. Revised Selected Papers*. *Lecture Notes in Computer Science*, vol. 8540, pp. 411–430. Springer (2014). https://doi.org/10.1007/978-3-662-46706-0_21, https://doi.org/10.1007/978-3-662-46706-0_21
9. Blondeau, C., Leander, G., Nyberg, K.: Differential-linear cryptanalysis revisited. *J. Cryptol.* **30**(3), 859–888 (2017). <https://doi.org/10.1007/S00145-016-9237-5>, <https://doi.org/10.1007/s00145-016-9237-5>
10. Broll, M., Canale, F., David, N., Flórez-Gutiérrez, A., Leander, G., Naya-Plasencia, M., Todo, Y.: New attacks from old distinguishers improved attacks on serpent. In: Galbraith, S.D. (ed.) *Topics in Cryptology - CT-RSA 2022 - Cryptographers' Track at the RSA Conference 2022, Virtual Event, March 1-2, 2022, Proceedings*. *Lecture Notes in Computer Science*, vol. 13161, pp. 484–510. Springer (2022). https://doi.org/10.1007/978-3-030-95312-6_20, https://doi.org/10.1007/978-3-030-95312-6_20
11. Daemen, J., Hoffert, S., Peeters, M., Assche, G.V., Keer, R.V.: Xoodyak, a lightweight cryptographic scheme. *IACR Trans. Symmetric Cryptol.* **2020**(S1), 60–87 (2020). <https://doi.org/10.13154/TOSC.V2020.IS1.60-87>, <https://doi.org/10.13154/tosc.v2020.is1.60-87>
12. Dobraunig, C., Eichlseder, M., Mendel, F., Schläffer, M.: Cryptanalysis of ascon. In: Nyberg, K. (ed.) *Topics in Cryptology - CT-RSA 2015, The Cryptographer's Track at the RSA Conference 2015, San Francisco, CA, USA, April 20-24, 2015. Proceedings*. *Lecture Notes in Computer Science*, vol. 9048, pp. 371–387. Springer (2015). https://doi.org/10.1007/978-3-319-16715-2_20, https://doi.org/10.1007/978-3-319-16715-2_20
13. Dobraunig, C., Eichlseder, M., Mendel, F., Schläffer, M.: Ascon v1.2: Lightweight authenticated encryption and hashing. *J. Cryptol.* **34**(3), 33 (2021). <https://doi.org/10.1007/s00145-021-09398-9>, <https://doi.org/10.1007/s00145-021-09398-9>
14. Dunkelman, O., Indestege, S., Keller, N.: A differential-linear attack on 12-round serpent. In: Chowdhury, D.R., Rijmen, V., Das, A. (eds.) *Progress in Cryptology - INDOCRYPT 2008, 9th International Conference on Cryptology in India, Kharagpur, India, December 14-17, 2008. Proceedings*. *Lecture Notes in Computer Science*, vol. 5365, pp. 308–321. Springer (2008). https://doi.org/10.1007/978-3-540-89754-5_24, https://doi.org/10.1007/978-3-540-89754-5_24
15. Dunkelman, O., Weizman, A.: Differential-linear cryptanalysis on xoodyak. *Lightweight Cryptography Workshop 2022* (2022), <https://csrc.nist.gov/Presentations/2022/differential-linear-cryptanalysis-on-xoodyak>
16. Flórez-Gutiérrez, A., Todo, Y.: Improving linear key recovery attacks using walsh spectrum puncturing. In: Joye, M., Leander, G. (eds.) *Advances in Cryptology - EUROCRYPT 2024 - 43rd Annual International Conference on the Theory and Applications of Cryptographic Techniques, Zurich, Switzerland, May 26-30, 2024, Proceedings, Part I*. *Lecture Notes in Computer Science*, vol. 14651, pp. 187–

216. Springer (2024). https://doi.org/10.1007/978-3-031-58716-0_7, https://doi.org/10.1007/978-3-031-58716-0_7
17. Hadipour, H., Derbez, P., Eichlseder, M.: Revisiting differential-linear attacks via a boomerang perspective with application to aes, ascon, clefia, skinny, present, knot, twine, warp, lblock, simeck, and SERPENT. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference*, Santa Barbara, CA, USA, August 18-22, 2024, *Proceedings, Part IV*. *Lecture Notes in Computer Science*, vol. 14923, pp. 38–72. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_2, https://doi.org/10.1007/978-3-031-68385-5_2
18. Hell, M., Johansson, T., Maximov, A., Meier, W.: The grain family of stream ciphers. In: Robshaw, M.J.B., Billet, O. (eds.) *New Stream Cipher Designs - The eSTREAM Finalists*, *Lecture Notes in Computer Science*, vol. 4986, pp. 179–190. Springer (2008). https://doi.org/10.1007/978-3-540-68351-3_14, https://doi.org/10.1007/978-3-540-68351-3_14
19. Hu, K., Peyrin, T., Tan, Q.Q., Yap, T.: Revisiting higher-order differential-linear attacks from an algebraic perspective. In: Guo, J., Steinfeld, R. (eds.) *Advances in Cryptology - ASIACRYPT 2023 - 29th International Conference on the Theory and Application of Cryptology and Information Security*, Guangzhou, China, December 4-8, 2023, *Proceedings, Part III*. *Lecture Notes in Computer Science*, vol. 14440, pp. 405–435. Springer (2023). https://doi.org/10.1007/978-981-99-8727-6_14, https://doi.org/10.1007/978-981-99-8727-6_14
20. Huang, T., Tjuawinata, I., Wu, H.: Differential-linear cryptanalysis of ICEPOLE. In: Leander, G. (ed.) *Fast Software Encryption - 22nd International Workshop, FSE 2015, Istanbul, Turkey, March 8-11, 2015, Revised Selected Papers*. *Lecture Notes in Computer Science*, vol. 9054, pp. 243–263. Springer (2015). https://doi.org/10.1007/978-3-662-48116-5_12, https://doi.org/10.1007/978-3-662-48116-5_12
21. Knellwolf, S., Meier, W., Naya-Plasencia, M.: Conditional differential cryptanalysis of nlfsr-based cryptosystems. In: Abe, M. (ed.) *Advances in Cryptology - ASIACRYPT 2010 - 16th International Conference on the Theory and Application of Cryptology and Information Security*, Singapore, December 5-9, 2010. *Proceedings*. *Lecture Notes in Computer Science*, vol. 6477, pp. 130–145. Springer (2010). https://doi.org/10.1007/978-3-642-17373-8_8, https://doi.org/10.1007/978-3-642-17373-8_8
22. Langford, S.K., Hellman, M.E.: Differential-linear cryptanalysis. In: Desmedt, Y. (ed.) *Advances in Cryptology - CRYPTO '94, 14th Annual International Cryptology Conference*, Santa Barbara, California, USA, August 21-25, 1994, *Proceedings*. *Lecture Notes in Computer Science*, vol. 839, pp. 17–25. Springer (1994). https://doi.org/10.1007/3-540-48658-5_3, https://doi.org/10.1007/3-540-48658-5_3
23. Leurent, G.: Improved differential-linear cryptanalysis of 7-round chaskey with partitioning. In: Fischlin, M., Coron, J. (eds.) *Advances in Cryptology - EUROCRYPT 2016 - 35th Annual International Conference on the Theory and Applications of Cryptographic Techniques*, Vienna, Austria, May 8-12, 2016, *Proceedings, Part I*. *Lecture Notes in Computer Science*, vol. 9665, pp. 344–371. Springer (2016). https://doi.org/10.1007/978-3-662-49890-3_14, https://doi.org/10.1007/978-3-662-49890-3_14
24. Li, J., Guan, J.: Advanced conditional differential attack on grain-like stream cipher and application on grain v1. *IET Inf. Secur.* **13**(2), 141–

- 148 (2019). <https://doi.org/10.1049/IET-IFS.2018.5180>, <https://doi.org/10.1049/iet-ifs.2018.5180>
25. Li, Z., Dong, X., Wang, X.: Conditional cube attack on round-reduced ASCON. *IACR Trans. Symmetric Cryptol.* **2017**(1), 175–202 (2017). <https://doi.org/10.13154/tosc.v2017.i1.175-202>, <https://doi.org/10.13154/tosc.v2017.i1.175-202>
 26. Liu, M., Lu, X., Lin, D.: Differential-linear cryptanalysis from an algebraic perspective. In: Malkin, T., Peikert, C. (eds.) *Advances in Cryptology - CRYPTO 2021 - 41st Annual International Cryptology Conference, CRYPTO 2021, Virtual Event, August 16-20, 2021, Proceedings, Part III. Lecture Notes in Computer Science*, vol. 12827, pp. 247–277. Springer (2021). https://doi.org/10.1007/978-3-030-84252-9_9, https://doi.org/10.1007/978-3-030-84252-9_9
 27. Lu, J.: A methodology for differential-linear cryptanalysis and its applications - (extended abstract). In: Canteaut, A. (ed.) *Fast Software Encryption - 19th International Workshop, FSE 2012, Washington, DC, USA, March 19-21, 2012. Revised Selected Papers. Lecture Notes in Computer Science*, vol. 7549, pp. 69–89. Springer (2012). https://doi.org/10.1007/978-3-642-34047-5_5, https://doi.org/10.1007/978-3-642-34047-5_5
 28. Matsui, M.: Linear cryptanalysis method for DES cipher. In: Hellesest, T. (ed.) *Advances in Cryptology - EUROCRYPT '93, Workshop on the Theory and Application of Cryptographic Techniques, Lofthus, Norway, May 23-27, 1993, Proceedings. Lecture Notes in Computer Science*, vol. 765, pp. 386–397. Springer (1993). https://doi.org/10.1007/3-540-48285-7_33, https://doi.org/10.1007/3-540-48285-7_33
 29. McLaughlin, J., Clark, J.A.: Filtered nonlinear cryptanalysis of reduced-round serpent, and the wrong-key randomization hypothesis. In: Stam, M. (ed.) *Cryptography and Coding - 14th IMA International Conference, IMACC 2013, Oxford, UK, December 17-19, 2013. Proceedings. Lecture Notes in Computer Science*, vol. 8308, pp. 120–140. Springer (2013). https://doi.org/10.1007/978-3-642-45239-0_8, https://doi.org/10.1007/978-3-642-45239-0_8
 30. Meltem Sönmez Turan, Kerry A. McKay, D.C.J.K.J.K.: Ascon-based lightweight cryptography standards for constrained devices. (National Institute of Standards and Technology, Gaithersburg, MD), NIST Special Publication (SP) NIST SP 800-232 ipd (2024), <https://doi.org/10.6028/NIST.SP.800-232.ipd>
 31. Nguyen, P.H., Wu, H., Wang, H.: Improving the algorithm 2 in multidimensional linear cryptanalysis. In: Paramalli, U., Hawkes, P. (eds.) *Information Security and Privacy - 16th Australasian Conference, ACISP 2011, Melbourne, Australia, July 11-13, 2011. Proceedings. Lecture Notes in Computer Science*, vol. 6812, pp. 61–74. Springer (2011). https://doi.org/10.1007/978-3-642-22497-3_5, https://doi.org/10.1007/978-3-642-22497-3_5
 32. Peng, T., Zhang, W., Weng, J., Ding, T.: New approaches for estimating the bias of differential-linear distinguishers. In: Reyzin, L., Stebila, D. (eds.) *Advances in Cryptology - CRYPTO 2024 - 44th Annual International Cryptology Conference, Santa Barbara, CA, USA, August 18-22, 2024, Proceedings, Part IV. Lecture Notes in Computer Science*, vol. 14923, pp. 174–205. Springer (2024). https://doi.org/10.1007/978-3-031-68385-5_6, https://doi.org/10.1007/978-3-031-68385-5_6
 33. Selçuk, A.A.: On probability of success in linear and differential cryptanalysis. *J. Cryptol.* **21**(1), 131–147 (2008). <https://doi.org/10.1007/S00145-007-9013-7>, <https://doi.org/10.1007/s00145-007-9013-7>

34. Xu, Z., Xu, H., Tan, L., Qi, W.: Differential-linear cryptanalysis of reduced round chacha. *IACR Trans. Symmetric Cryptol.* **2024**(2), 166–189 (2024). <https://doi.org/10.46586/TOSC.V2024.I2.166-189>, <https://doi.org/10.46586/tosc.v2024.i2.166-189>
35. Zhou, H., Li, Z., Dong, X., Jia, K., Meier, W.: Practical key-recovery attacks on round-reduced ketje jr, xoodoo-ae and xodyak. *Comput. J.* **63**(8), 1231–1246 (2020). <https://doi.org/10.1093/COMJNL/BXZ152>, <https://doi.org/10.1093/comjnl/bxz152>

A The Differential-Linear Distinguishers of Serpent

In Table 10, we summarize the DL distinguishers for the 9-round Serpent, including the ones given in [17,32] and the new distinguishers we found. We rank all distinguishers according to the experimental bias, from largest to smallest.

Table 10. DL distinguishers for 9-round Serpent

the differential-linear distinguisher	Theor. bias	Exper. bias
offset = 1, $r_0 = 1, r_m = 3, r_1 = 5, p = 2^{-5}, r = 2^{-8.69}, q = 2^{-20}$		
Δ_i 00001009000000000000000000000000 Δ_o 00000000000000400000000000000000	$2^{-51.69}$	$2^{-51.45}$
λ_i 400000000000000000000000000000200 λ_o 20a8010212f00a32000b0020b4010b02		
Δ_i 00000100900000000000000000000000 Δ_o 00000000000000400000000000000000		
λ_i 040000000000000000000000000000020 λ_o 220a8000210d00a32020b4020a401010		
Δ_i 00000010090000000000000000000000 Δ_o 00000000000000400000000000000000		
λ_i 004000000000000000000000000000002 λ_o 0220a0000200500032000b0020b4010b		
offset = 1, $r_0 = 2, r_m = 3, r_1 = 4, p = 2^{-7}, r = 2^{-13.96}, q = 2^{-16}$		
Δ_i 0000000000000000200a000000000000 Δ_o 00000000000000100008100002004400	$2^{-50.96}$ ($2^{-51.95}$ [17])	$2^{-51.89}$
λ_i 000000000000000000000000800000000 λ_o 2a00308043090ab2e02a240400801089		
offset = 1, $r_0 = 1, r_m = 3, r_1 = 5, p = 2^{-4}, r = 2^{-9.96}, q = 2^{-20}$		
Δ_i c00000000000090000000000000000000 Δ_o 00000000000000000000000000000000	$2^{-51.96}$	$2^{-51.99}$
λ_i 004000000000000000000000000000002 λ_o 0220a0000200500032000b0020b4010b		
offset = 1, $r_0 = 2, r_m = 2, r_1 = 5, p = 2^{-7}, r = 2^{-7}, q = 2^{-20}$		
Δ_i 00000100a00000000000000000000000 Δ_o 000000000000000010000810000200440	2^{-52} (2^{-52} [32])	2^{-52}
λ_i 004000000000000000000000000000002 λ_o 92a00308043090ab2e02a24040080108		
offset = 2, $r_0 = 1, r_m = 3, r_1 = 5, p = 2^{-4}, r = 2^{-7.26}, q = 2^{-23}$		
Δ_i 00000000040070000000000000000000 Δ_o 00000000000000000000400000000000	$2^{-55.26}$ ($2^{-55.33}$ [32])	$2^{-55.33}$
λ_i 000000000000000000000000800000000 λ_o 00b0000b000030000b0200e000000100		
Δ_i 00000000004007000000000000000000 Δ_o 00000000000000000000400000000000		
λ_i 000000000000000000000000800000000 λ_o 000b0000b000030000b0200e00000010		
offset = 2, $r_0 = 2, r_m = 3, r_1 = 4, p = 2^{-7}, r = 2^{-8.4}, q = 2^{-21}$		
Δ_i 00000000000000000000000000004007 Δ_o 1000020004000000000000000000100008	$2^{-55.40}$ ($2^{-55.43}$ [17])	$2^{-55.42}$
λ_i 000000000000000010000a000000000000 λ_o 000048110818a00a3004038010200a04		
offset = 1, $r_0 = 2, r_m = 2, r_1 = 5, p = 2^{-7}, r = 2^{-12}, q = 2^{-20}$		
Δ_i 0000000000000000200a000000000000 Δ_o 000000000000000010000810000200440	2^{-57} (2^{-52} [32])	$2^{-56.98}$
λ_i 040000000000000000000000000000020 λ_o 2a00308043090ab2e02a24040080108a		

B The Difference Part of the 10-round Serpent Distinguisher

We depict the differential part of the 10-round DL distinguisher in Fig. 5. In the description of differentials a thin arrow represents a probability of 1/8 for the specific S-box (given the input difference, the output difference is achieved with probability 1/8), and a fat arrow stands for probability 1/4. If there is a difference in a bit, its entry is filled.

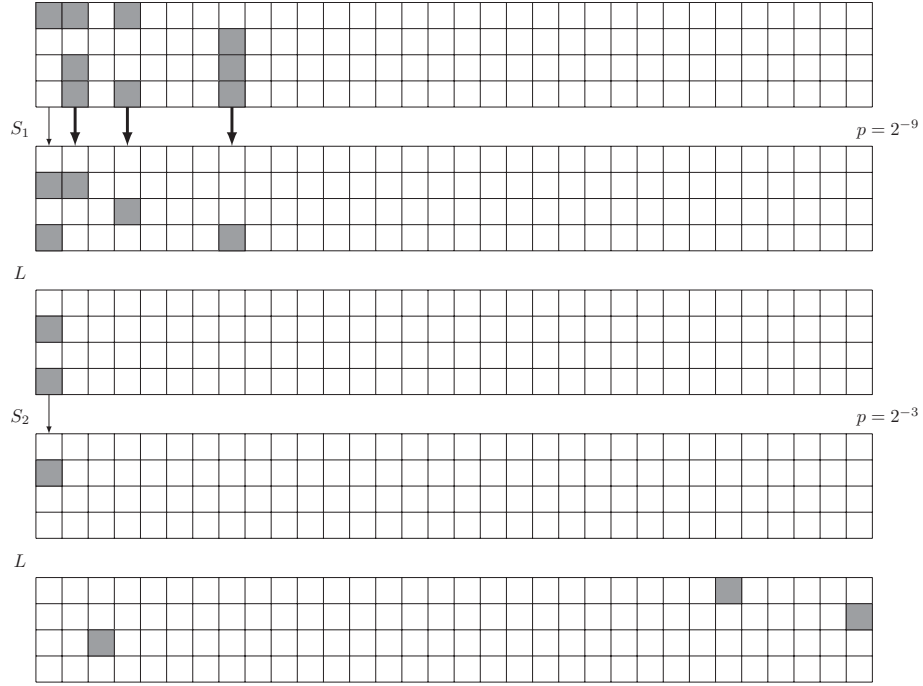


Fig. 5. The difference part of the 10-round distinguisher

C The Conditions in the 6-Round Attack on Xoodyak

$$\begin{aligned}
 v_4 &= k_{118} + k_{127} + v_{118} + v_{127} + v_{246} + v_{255} \\
 v_{10} &= k_{10} + k_{19} + v_{19} + v_{138} + v_{147} + v_{184} \\
 v_{11} &= k_{11} + k_{20} + v_{20} + v_{139} + v_{148} + v_{185} + 1 \\
 v_{21} &= k_{21} + k_{30} + k_{35} + v_{30} + v_{149} + v_{158} + 1 \\
 v_{82} &= k_{82} + k_{91} + v_{91} + v_{96} + v_{210} + v_{219} \\
 v_{102} &= k_{102} + k_{111} + v_{111} + v_{148} + v_{230} + v_{239} \\
 v_{103} &= k_{103} + k_{112} + v_{112} + v_{149} + v_{231} + v_{240} + 1 \\
 v_{113} &= k_{31} + k_{113} + k_{122} + v_{122} + v_{241} + v_{250} + 1
 \end{aligned}$$

D The Conditions in the 126-Round Attack on Grain v1

We impose 11 Type 0 conditions and 14 Type 1 conditions.

Firstly, we list Type 0 conditions

$$\begin{aligned}
 v_4 = 1, v_9 = 0, v_{25} = 0, v_{26} = 0, v_{31} = 1, v_{50} = 0, \\
 v_{53} = 0, v_{59} = 0, v_{61} = 0, v_{62} = 0, v_{63} = 0.
 \end{aligned}$$

Then, we give Type 1 conditions in order of occurrence.

At round 19, we impose the condition

$$v_2 + v_5 v_{27} v_{48} + v_5 v_{48} k_{65} + v_5 v_{48} + v_5 + v_{27} v_{48} k_{65} + v_{27} + v_{44} + v_{48} k_{65} + v_{48} + f_1(K) = 0,$$

where $f_1(K)$ is a function of key bits only.

At round 22, we impose the condition

$$v_5 + v_8 v_{30} v_{51} + v_8 v_{51} k_{68} + v_8 v_{51} + v_8 + v_{30} v_{51} k_{68} + v_{30} + v_{51} k_{68} + v_{51} + f_2(K) = 0,$$

where $f_2(K)$ is a function of key bits only.

At round 40, we impose the conditions

$$v_6 + v_{52} + f_3(K) = 0,$$

$$v_{43} + v_{23} + v_{48} + f_4(K) = 0,$$

$$v_6 + v_{19} + v_{29} + v_{44} + v_{52} + v_{57} + f_5(K) = 0,$$

$$v_3 + v_6 v_{28} v_{49} + v_6 v_{49} k_{66} + v_6 v_{49} + v_6 + v_{28} v_{49} k_{66} + v_{28} + v_{49} k_{66} + v_{49} + f_6(K) = 0,$$

where $f_3(K), f_4(K), f_5(K), f_6(K)$ are functions of key bits only.

At round 44, we impose the conditions

$$\begin{aligned} v_{10} + v_{13} v_{35} v_{56} + v_{13} v_{56} k_{73} + v_{13} v_{56} + v_{13} + v_{23} + v_{33} + v_{35} v_{56} k_{73} \\ + v_{35} + v_{48} + v_{56} k_{73} + v_{56} + f_7(K) = 0, \end{aligned}$$

$$\begin{aligned} v_{12} + v_{15} v_{37} v_{58} + v_{15} v_{58} k_{75} + v_{15} v_{58} + v_{15} + v_{28} + v_{35} + v_{37} v_{58} k_{75} \\ + v_{37} + v_{41} + v_{51} + v_{58} k_{75} + v_{58} + f_8(K) = 0, \end{aligned}$$

where $f_7(K), f_8(K)$ are functions of key bits only.

At round 50, we impose the conditions

$$v_{16} + v_{41} + f_9(K) = 0,$$

$$v_{13} + v_{16} + v_{38} + f_{10}(K) = 0,$$

$$v_1 + v_{14} + v_{24} + v_{39} + v_{52} + f_{11}(K) = 0,$$

$$v_{16} + v_{29} + v_{39} + v_{41} + v_{54} + f_{12}(K) = 0,$$

$$v_0 + v_3 v_{46} k_{63} + v_3 v_{46} + v_3 + v_{13} + v_{23} + v_{38} + v_{46} k_{63} + v_{46} + v_{51} + f_{13}(K) = 0,$$

$$\begin{aligned} v_{12} v_{34} v_{55} + v_{12} v_{55} k_{72} + v_{12} v_{55} + v_{12} + v_{15} + v_{17} + v_{18} + v_{20} + v_{30} \\ + v_{33} + v_{34} v_{55} k_{72} + v_{34} + v_{36} v_{58} + v_{42} + v_{55} k_{72} + v_{58} + f_{14}(K) = 0, \end{aligned}$$

where $f_9(K), f_{10}(K), f_{11}(K), f_{12}(K), f_{13}(K), f_{14}(K)$ are functions of key bits only.